

COMP3018: Report (Literature Review & Programming Project)

Cognitive Robotics

Contents

1- Task (3) Literature Review	1
[] 1.3.1 Therapeutic and Emotional Support	3
[] 1.3.2 Medication Adherence and Daily Living Support	3
[] 1.3.3 Physical Rehabilitation and Mobility	4
[] 1.4.1 Challenges	4
[] 1.4.2 Ethical Implications	5
1.6. Task-3 References	6
1.6.1 Papers and Journal Articles	6
2- Task (4) Novel Programming Project (Adaptiveness in Assistive Robotics)	8
2.1. Introduction (10%; Salman)	8
2.2. Background (10%; Alfie)	8
2.3. Methods & Setup (35%; Alfie)	9
2.3.1 System Architecture	9
2.3.2 Input Layer: Multimodal and Behavioural Signals	9
2.3.3 Process Layer: Multi-Signal State Inference	13
2.3.4 Generate Layer: Dynamic Prompt Construction	16
2.3.5 Output Layer: Aligned Multimodal Response	17
2.3.6 Adaptation Self-Evaluation and Session Persistence	17
2.4. Outcome & System Analysis (30%; Salman)	17
2.5. Conclusion (10%; Alfie)	17
2.6 Task-4 References (5%)	17
3. Appendix	18
3.1 Video Demo	18
3.2 AI Declaration	19
3.3 gaze22.py	20

1- Task (3) Literature Review

☒ [X] ## 1.1. Introduction

Ageing populations and a shrinking care workforce positioned **assistive robotics** (*human-supportive robots within physical, cognitive, and social realms of impairment affecting daily-living activities*); a prominent technological response to a widening care gap. The field spans diverse subfields; this essay however focuses on *socially* assistive robotics (*SAR*), an assistive-robot subfield wherein the robot “focuses on helping human users through social rather than physical interaction” (Tapus, Matarić and Scassellati, 2007, Abstract), as here most active research and ethical tension converge.

Robots now administer medication reminders, facilitate rehabilitation exercises, and therapeutically companionate in clinical and domestic settings: reduced caregiver burden, improved patient outcomes wherein

residents became “more active and more communicative, both with each other and their caregivers” (Wada and Shibata, 2007, p. 973).

However, most-current assistive robots operate at what Sciutti et al. (2023, p. 160) call the social layer: they react to immediate stimuli but lack the cognitive depth to anticipate user needs, remember past interactions, or reason about their own performance. Sciutti et al. argue that effective assistive robots should be *cognitive*: capable of “flexible, context-sensitive action, knowing what they are doing and why they are doing it.” Vernon, Metta and Sandini (2007, p. 151) formalise this requirement via a “virtuous cycle that is embedded in an ongoing process of action and perception” (the agent anticipates → learns → adapts to achieve autonomy). This essay contends that assistive robotics should graduate from reactive social behaviour to cognitive capability (intelligence deployed *over* the social layer) for sustained, personalised support. The following sections survey theoretical foundations thereof, applications, challenges, and future directions.

[X] ## 1.2. Literature Review

Cognitive robotics: defined by Sandini, Sciutti and Vernon (2021, Overview), lies at the intersection of Robotics, Artificial Intelligence, and Cognitive and Biological Sciences, integrating “sensorimotor skills, knowledge representation and reasoning, and social interaction.” This interdisciplinary grounding distinguishes it from conventional robotics (*treats the robot as purely engineered*) and from social robotics (*addresses interaction behaviour without necessarily modelling cognitive processes*). The distinction is consequential: a robot that smiles when a patient smiles is social; a robot that infers *why*, and adapts accordingly, is *cognitive*.

□ verify following lecture slides

The European Network for Advancement of Artificial Cognitive Systems (euCognition) catalogued 42 definitions of cognition, yet the common thread therein: anticipation, learning, and adaptation, intersected with perception and action to create autonomy (Fig. 1). This cycle provides an architectural checklist for assistive robots: a system that cannot direct its gaze toward relevant stimuli whilst suppressing irrelevant ones (*selective attention*), anticipate the outcome of its actions (*i.e. prospection*), learn from past interactions (*memory*), or adapt its strategy when performance declines (*metacognition*) is, per this framework, not yet cognitive. Kotseruba and Tsotsos’s (2020, Abstract) survey of 84 cognitive architectures organises them along precisely these abilities, thus corroborating the checklist’s standing in the field. Sciutti et al. (2023, p. 160) further specify that cognitive robots should “reason about their actions and modify their behavior to improve their effectiveness”; a capacity termed *theory of mind*, wherein the agent infers another’s hidden mental state.

Furthermore, memory is not monolithic (*treated as a single uniform store*). Laird (2012, Chapter 9) distinguishes *episodic memory* (*records of specific past experiences and their contextual outcomes*) from *semantic memory* (*general knowledge about the world, including spatial relationships and factual constraints*) in implementational terms: episodic memory is “what you ‘remember’” whilst semantic memory is “what you ‘know’”. For example, assistive-medication robots need episodic memory to recall that a user refused medication after a restless night, and semantic memory to know certain drugs cannot also be administered.

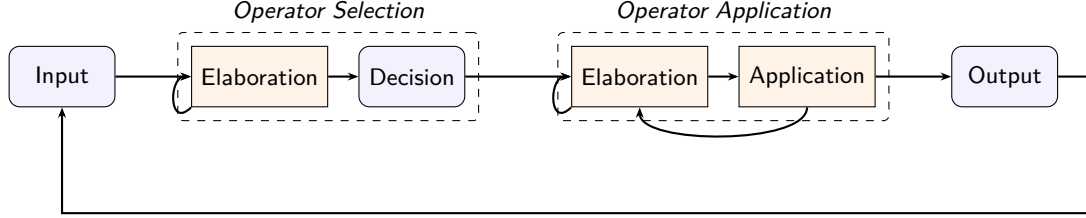


Figure 1: Soar’s processing cycle, adapted from Laird (2012, fig. 4.7, p. 79), as a concrete instantiation of Vernon, Metta and Sandini’s (2007) cognition cycle. Laird specifies that the cycle “consists of four phases: Input, Operator Selection, Operator Application, and Output” (Laird, 2012, p. 79), wherein Input maps to perception, Operator Selection draws on episodic and semantic memory to anticipate consequences (i.e. *prospection*), Operator Application executes the chosen behaviour through parallel rule-firing waves, and Output returns the agent to the environment whilst the loop hands control back to Input for continual adaptation. This architecture exposes the deficit of reactive systems: PARO, for instance, implements only Input $\rightarrow$ Output, lacking the substate-driven Operator Selection wherein deliberation over latent user states could occur. Furthermore, Laird argues that “the most significant effect of chunking is that it eliminates processing in substates for situations similar to ones experienced in the past” (Laird, 2012, p. 164), wherein repeated deliberations compile into procedural rules; a mechanism distinct from POMDP belief-space approximation, yet complementary thereto in addressing real-time embodied operation under care-time constraints.

□ ## 1.3. Applications

[] 1.3.1 Therapeutic and Emotional Support

The PARO therapeutic seal robot is among the most-widely deployed platforms within socially assistive robotics (Tapus, Mataric and Scassellati, 2007, .pdf-p. 1). Wada and Shibata (2007, p. 973) demonstrate that PARO improves mood in patients with dementia, utilising tactile and auditory inputs. Clinical trials report that urinary stress indicators “significantly improved” after PARO’s introduction (Wada and Shibata, 2007, p. 978, Table II), therefore the platform has been adopted in care homes.

Notwithstanding the benefits PARO operates at the reactive layer, possesses no theory of mind (cannot infer *why* a patient is agitated (loneliness, pain, confusion) nor episodic memory of *what* calmed the patient previously. A cognitively-equipped therapeutic robot, in contrast, would anticipate mood shifts via *prospection* $\rightarrow$ recall that music soothed this patient yesterday via episodic memory $\rightarrow$ adapt its strategy via metacognition. PARO’s effectiveness plateaus precisely because it cannot personalise; the gap is therefore clinically consequential. The architectural cost is precise: without episodic memory, the agent’s perception is “limited both spatially and temporally—that is, to the here and now” (Laird, 2012, p. 233). Fong, Nourbakhsh and Dautenhahn (2003, p. 145) formalise this gap via Breazeal’s taxonomy: PARO occupies the “social interface” level (human-like cues but “shallow models of social cognition”), whereas Sciutti et al.’s (2023, p. 160) vision of robots “knowing what they are doing and why” demands the *socially intelligent* level.

[] 1.3.2 Medication Adherence and Daily Living Support

Medication non-adherence imposes substantial costs on healthcare systems, and elderly patients with polypharmacy regimens are particularly vulnerable to missed or incorrect doses. Robots in this domain face a different challenge from therapeutic companionship: trust and cognitive load are latent variables, inferred only from noisy behavioural proxies. Lee and See (2004, .pdf-p. 6) define trust as “the attitude that an agent will help achieve an individual’s goals in a situation characterized by uncertainty and vulnerability”; a definition foregrounding the unobservable nature that necessitates probabilistic modelling. A user may comply with a medication prompt despite low trust (e.g. time pressure), or indeed refuse despite high trust (e.g. task complexity), and thus the observation alone cannot reliably disambiguate the underlying state (Kaelbling, Littman and Cassandra, 1998, p. 105, 3. *Partial observability*).

The Partially Observable Markov Decision Process (POMDP) provides formal machinery for this uncertainty. Chen et al. (2020, p. 6) demonstrate a Trust-POMDP wherein the robot maintains a belief distribution over

trust and selects actions that maximise long-term collaboration, showing belief-space planning outperforms fixed strategies. Garcez and Lamb (2023, *.pdf*-p. 1) identify the neuro-symbolic paradigm as the ‘third wave’ of AI, wherein neural subsystems (LLMs) handle perception whilst symbolic subsystems (e.g. POMDPs) govern temporal reasoning. Nikolaidis, Hsu and Srinivasa (2017, p. 625) provide empirical corroboration: in a collaborative task ($n = 69$), robots utilising mutual adaptation via a Mixed Observability MDP (modelling human adaptability as a latent variable) were rated significantly more trustworthy than fixed-policy alternatives ($U = 180$, $p = 0.048$). This aligns with Hancock et al.’s (2011, p. 522) finding that performance is the strongest trust predictor.

[] 1.3.3 Physical Rehabilitation and Mobility

Robotic exoskeletons and assistive manipulators for stroke recovery and mobility support need to adapt not only to the patient’s physical state (joint angles and muscle activation patterns) but also to their psychological state: motivation, frustration, and fatigue determine whether a patient perseveres or disengages.

Embodied cognition becomes essential. Brooks (1991, Introduction) argues that intelligence emerges from physical interaction with the environment rather than abstract representation, whereas Fong, Nourbakhsh and Dautenhahn (2003, p. 149) operationalise this as “perturbatory coupling”: the more channels of mutual influence, the more embodied the system. A rehabilitation robot therefore occupies a uniquely cognitive niche, sensing the patient’s body, reasoning about current capabilities, and adapting appropriately. A purely language-based or screen-based interface cannot achieve this; Matarić et al. (2007, *.pdf*-p. 7) confirm: stroke survivors engaged more with embodied robots than screen-based alternatives. Tapus, Țăpuș and Matarić (2008, Abstract) show that embodiment alone is insufficient: adaptive personality matching (adjusting interaction distance and speed to the user’s traits) further improved task performance, suggesting rehabilitation robots require physical presence and cognitive adaptation. - [] ## 1.4. Discussion

[] 1.4.1 Challenges

Tapus, Matarić and Scassellati (2007, *.pdf*-p. 6) projected that by 2012 SAR systems would demonstrate “marked improvement in learning/training/recovery of the user”; yet PARO, the most-deployed platform nearly twenty years later, *still* cannot remember yesterday’s session. Three challenges explain this. Firstly, computational intractability: solving *POMDPs* exactly is PSPACE-complete (Papadimitriou and Tsitsiklis, 1987, p. 448), and the belief simplex grows exponentially with state-space dimensionality. Whilst approximate solvers such as point-based value iteration (Pineau, Gordon and Thrun, 2003, p. 1025) and online Monte-Carlo tree search (Silver and Veness, 2010, p. 1) mitigate this, real-time cognitive processing within embodied systems remains an open challenge, particularly when multiple unobserved variables (trust, load, emotion) require tracking simultaneously.

Secondly, the measurement problem: trust, cognitive load, and emotional state are not directly observable; observations thereof are noisy proxies at best. Hancock et al.’s (2011, p. 522) meta-analysis of 29 studies finds that even the strongest correlates of trust explain only moderate variance, whilst Broadbent, Stafford and MacDonald (2009, Abstract) note that acceptance depends on matching robot behaviour to user expectations, not trust alone. Desai et al. (2013) demonstrate trust dynamics are non-linear, building slowly through consistent performance but degrading rapidly after errors; and thus a single misclassified observation can cascade into wrong actions. Nikolaidis, Hsu and Srinivasa (2017, p. 626), however, demonstrate that mutual adaptation partially mitigates this fragility: when the robot models human adaptability as a latent variable, trust persists even during strategy disagreements, suggesting the variance Hancock et al. report may stem from studies that treat the human as a static rather than co-adaptive partner.

Finally, adaptation without exploitation: a robot that runs inference on cognitive load could time its medication requests to coincide with periods of high vulnerability, maximising compliance at the expense of user autonomy. The POMDP reward function should therefore encode ethical constraints alongside clinicians’ objectives.

[] 1.4.2 Ethical Implications

Assistive robots in intimate care spaces (bedrooms, bathrooms, rehabilitation clinics) continuously collect sensitive behavioural data. Facial expressions, vocal patterns, and movement trajectories constitute biometric data, yet regulatory frameworks have not kept pace with deployment. Wachter, Mittelstadt and Floridi (2017, Abstract) argue that even the General Data Protection Regulation provides no enforceable “right to explanation” of automated decisions; a gap particularly concerning in healthcare wherein recommendations directly affect patient wellbeing.

Moreover, over-reliance on assistive robots risks eroding functional independence. If a robot anticipates and pre-empts needs via prospection, the user may disengage from self-directed activity, contradicting the assistive mandate. Sharkey (2014, *.pdf*-p. 6) frames this via Nordenfelt’s ‘Dignity of Identity’: “a robot that dealt impersonally with an older person, without knowing or using their name or their preferences would also be likely to negatively affect their feelings of dignity.” This implies that only cognitively-equipped robots (with episodic memory of individuals) can avoid dignity violations; reactive systems such as PARO, regardless of their therapeutic benefits (Wada and Shibata, 2007, p. 973), risk infantilisation precisely because they cannot personalise. The responsibility gap compounds this further: when a care robot administers incorrect medication, liability falls ambiguously between manufacturer, deployer, and clinician.

Furthermore, the deployment of assistive robots is not equitable: wealthy nations stand to benefit, whilst low-income populations face a widening digital divide. Whether they displace or augment carers remains unresolved.

The ethical watchword is therefore proactive regulation: design-stage ethics anticipating failure modes, not reactive patchwork after harm. Per the embodied cognition thesis, if intelligence indeed requires a body, and that body enters the most intimate spaces of vulnerable persons, then the ethical stakes of assistive cognitive robotics are uniquely high.

□ ## 1.5. Conclusion

Assistive robotics stands at an inflection point. Current systems deliver measurable benefits within narrow envelopes, yet their reactive architectures limit sustained, personalised effectiveness. The Vernon, Metta and Sandini (2007) cognition cycle provides the architectural blueprint for graduating beyond this plateau: assistive robots that anticipate (prospection), remember (episodic and semantic memory), reason about others’ mental states (theory of mind), and monitor their own performance (metacognition) would mark a qualitative advance over current systems.

The neuro-symbolic paradigm offers a viable path toward this vision, as the Trust-POMDP framework attests (Chen et al., 2020). Sciutti et al. (2023, p. 161) independently identify the integration of learning with model-based approaches as cognitive robotics’ most-prominent trajectory; that this converges with Garcez and Lamb’s (2023, *.pdf*-p. 1) ‘third wave’ thesis suggests the direction is robust rather than parochial. Future applications will likely extend beyond single-task assistance toward cognitively autonomous home-dwelling companions: robots that proactively monitor health indicators, anticipate daily needs via episodic memory, and adapt their interaction style to the user’s evolving state. Sharkey and Sharkey (2012, *.pdf*-p. 1) caution that such systems risk replacing rather than supplementing human-care; the field should therefore pursue cognitive capability and ethical governance in concert, lest the technology displace these very carers it was meant to supplement. Figure~2 visualises this trajectory. The ultimate test, per the embodied cognition thesis, is a robot that can: sense → remember → anticipate → adapt within the physical world, whilst respecting the autonomy, and dignity, of the persons it serves.

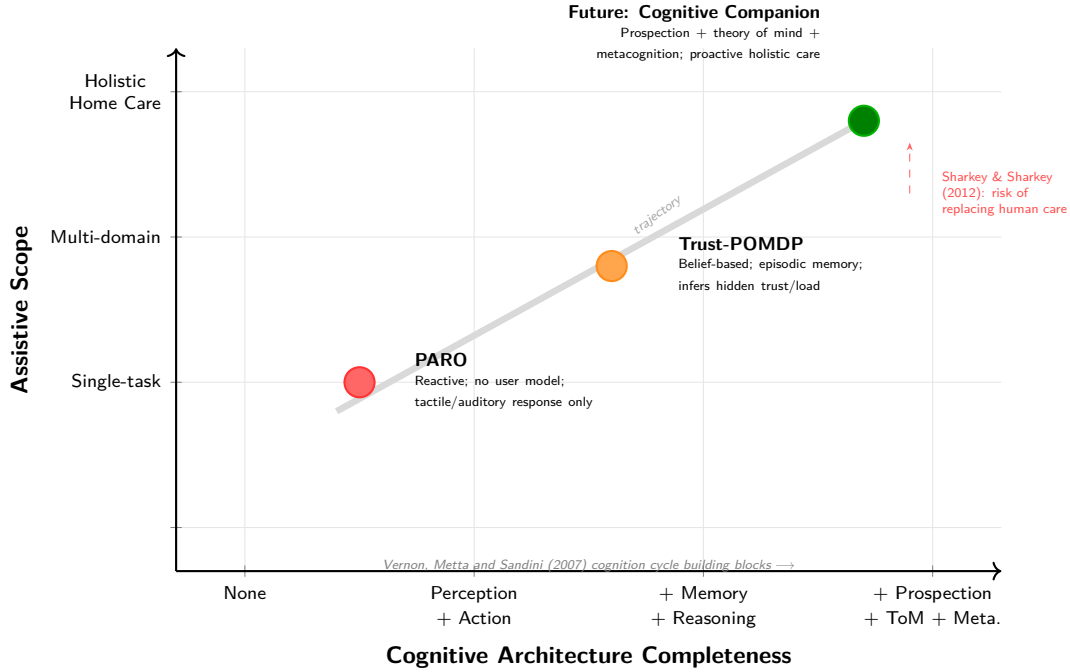


Figure 2: Trajectory of assistive robotics from reactive single-task systems (PARO) through adaptive belief-based architectures (Trust-POMDP) toward cognitively autonomous home-dwelling companions. Expanding assistive scope without expanding cognitive capability is insufficient; the diagonal trajectory requires both.

1.6. Task-3 References

1.6.1 Papers and Journal Articles

- ☒ verify all links work and extract each papers insights

REMOVE: one checkmark for working link second for insight gathered

- ☒ Broadbent, E., Stafford, R. and MacDonald, B. (2009) ‘Acceptance of Healthcare Robots for the Older Population: Review and Future Directions’, *International Journal of Social Robotics*, 1(4), pp. 319-330. Available at: https://www.researchgate.net/publication/220397395_Acceptance_of_Healthcare_Robots_for_the_Older_Population (Accessed: 25 March 2026).
- ☒ Brooks, R. A. (1991) ‘Intelligence without representation’, *Artificial Intelligence*, 47(1-3), pp. 139-159. Available at: <https://people.csail.mit.edu/brooks/papers/representation.pdf> (Accessed: 24 March 2026).
- ☒ Chen, M., Nikolaidis, S., Soh, H., Hsu, D. and Srinivasa, S. (2020) ‘Trust-Aware Decision Making for Human-Robot Collaboration: Model Learning and Planning’, *ACM Transactions on Human-Robot Interaction*, 9(2), pp. 1-23. Available at: <https://personalrobotics.cs.washington.edu/publications/chen2019trust.pdf> (Accessed: 15 March 2026).
- ☒ [] Desai, M., Kaniarasu, P., Medvedev, M., Steinfeld, A. and Yanco, H. (2013) ‘Impact of robot failures and feedback on real-time trust’, *Journal of Human-Robot Interaction*, 2(1), pp. 251-275. Available at: <https://ieeexplore.ieee.org/document/6483596> (Accessed: 20 March 2026).
- ☒ [] Fong, T., Nourbakhsh, I. and Dautenhahn, K. (2003) ‘A survey of socially interactive robots’, *Robotics and Autonomous Systems*, 42(3-4), pp. 143-166. Available at: <https://www.cs.cmu.edu/~illah/PAPERS/social.pdf> (Accessed: 18 March 2026).
- ☒ [] Garcez, A. d’A. and Lamb, L. C. (2023) ‘Neurosymbolic AI: The 3rd Wave’, *Artificial Intelligence Review*, 56, pp. 12387-12406. Available at: <https://link.springer.com/article/10.1007/s10462-023-10448-w> (Accessed: 20 March 2026).
- ☒ [] Hancock, P. A., Billings, D. R., Schaefer, K. E., Chen, J. Y. C., de Visser, E. J. and Parasuraman, R.

- (2011) ‘A meta-analysis of factors affecting trust in human-robot interaction’, *Human Factors*, 53(5), pp. 517-527. Available at: <https://journals.sagepub.com/doi/10.1177/0018720811417254> (Accessed: 15 March 2026).
- ⊗ [] Kaelbling, L. P., Littman, M. L. and Cassandra, A. R. (1998) ‘Planning and acting in partially observable stochastic domains’, *Artificial Intelligence*, 101(1-2), pp. 99-134. Available at: <https://people.csail.mit.edu/lpk/papers/aij98-pomdp.pdf> (Accessed: 13 March 2026).
 - ⊗ [] Kotseruba, I. and Tsotsos, J. K. (2020) ‘40 years of cognitive architectures: core cognitive abilities and practical applications’, *Artificial Intelligence Review*, 53(1), pp. 17-94. Available at: <https://link.springer.com/article/10.1007/s10462-018-9646-y> (Accessed: 3 May 2026).
 - ⊗ [] Laird, J. E. (2012) *The Soar Cognitive Architecture*. Cambridge, MA: MIT Press.
 - ⊗ [] Lee, J. D. and See, K. A. (2004) ‘Trust in automation: Designing for appropriate reliance’, *Human Factors*, 46(1), pp. 50-80. Available at: https://journals.sagepub.com/doi/10.1518/hfes.46.1.50_30392 (Accessed: 15 March 2026).
 - ⊗ [] Matarić, M. J., Eriksson, J., Feil-Seifer, D. J. and Winstein, C. J. (2007) ‘Socially assistive robotics for post-stroke rehabilitation’, *Journal of NeuroEngineering and Rehabilitation*, 4(5), pp. 1-9. Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC1821334/> (Accessed: 25 March 2026).
 - ⊗ [] Nikolaidis, S., Hsu, D. and Srinivasa, S. (2017) ‘Human-robot mutual adaptation in collaborative tasks: Models and experiments’, *The International Journal of Robotics Research*, 36(5-7), pp. 618-634. Available at: <https://journals.sagepub.com/doi/10.1177/0278364917690593> (Accessed: 20 March 2026).
 - ⊗ [] Papadimitriou, C. H. and Tsitsiklis, J. N. (1987) ‘The complexity of Markov decision processes’, *Mathematics of Operations Research*, 12(3), pp. 441-450. Available at: <https://web.mit.edu/jnt/www/Papers/J016-87-mdp-complexity.pdf> (Accessed: 13 March 2026).
 - ⊗ [] Pineau, J., Gordon, G. and Thrun, S. (2003) ‘Point-based value iteration: An anytime algorithm for POMDPs’, in *Proceedings of the 18th International Joint Conference on Artificial Intelligence (IJCAI-03)*, pp. 1025-1030. Available at: <http://www.cs.cmu.edu/~ggordon/jpineau-ggordon-thrun.ijcai03.pdf> (Accessed: 24 March 2026).
 - ⊗ [] Sandini, G., Sciutti, A. and Vernon, D. (2021) ‘Cognitive Robotics’, in *Encyclopedia of Robotics*. Berlin: Springer-Verlag. Available at: http://www.vernon.eu/publications/2021_Sandini_et_al.pdf (Accessed: 3 May 2026).
 - ⊗ [] Sciutti, A., Beetz, M., Inamura, T., Korsah, A., Oh, J., Sandini, G., Shimoda, S. and Vernon, D. (2023) ‘The Present and the Future of Cognitive Robotics’, *IEEE Robotics & Automation Magazine*, 30(3), pp. 160-163. Available at: <https://ieeexplore-ieee-org.plymouth.idm.oclc.org/document/10255092> (Accessed: 18 March 2026).
 - ⊗ [] Sharkey, A. (2014) ‘Robots and human dignity: A consideration of the effects of robot care on the dignity of older people’, *Ethics and Information Technology*, 16(1), pp. 63-75. Available at: <https://philarchive.org/rec/SHARAH-2> (Accessed: 22 March 2026).
 - ⊗ [] Sharkey, A. and Sharkey, N. (2012) ‘Granny and the robots: ethical issues in robot care for the elderly’, *Ethics and Information Technology*, 14(1), pp. 27-40. Available at: <https://philarchive.org/rec/SHAGAT> (Accessed: 22 March 2026).
 - ⊗ [] Silver, D. and Veness, J. (2010) ‘Monte-Carlo planning in large POMDPs’, in *Advances in Neural Information Processing Systems (NeurIPS 23)*, pp. 2164-2172. Available at: <https://proceedings.neurips.cc/paper/2010/file/Paper.pdf> (Accessed: 24 March 2026).
 - ⊗ [] Tapus, A., Matarić, M. J. and Scassellati, B. (2007) ‘Socially assistive robotics [Grand Challenges of Robotics]’, *IEEE Robotics & Automation Magazine*, 14(1), pp. 35-42. Available at: <https://scazlab.yale.edu/sites/default/files/files/Tapus-RAM2007.pdf> (Accessed: 25 March 2026).
 - ⊗ [] Tapus, A., Țăpuș, C. and Matarić, M. J. (2008) ‘User-robot personality matching and assistive robot behavior adaptation for post-stroke rehabilitation therapy’, *Intelligent Service Robotics*, 1(2), pp. 169-183. Available at: <https://hal.science/hal-00770108/document> (Accessed: 26 March 2026).
 - ⊗ [] Vernon, D., Metta, G. and Sandini, G. (2007) ‘A Survey of Artificial Cognitive Systems: Implications for the Autonomous Development of Mental Capabilities in Computational Agents’, *IEEE Transactions on Evolutionary Computation*, 11(2), pp. 151-180. Available at: https://www.robotcub.org/misc/papers/07_Vernon_Metta_Sandini_IEEE.pdf (Accessed: 13 March 2026).
 - ⊗ [] Wachter, S., Mittelstadt, B. and Floridi, L. (2017) ‘Why a Right to Explanation of Automated

Decision-Making Does Not Exist in the General Data Protection Regulation’, *International Data Privacy Law*, 7(2), pp. 76-99. Available at: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2903469 (Accessed: 22 March 2026).

- ☒ [] Wada, K. and Shibata, T. (2007) ‘Living with seal robots: its sociopsychological and physiological influences on the elderly at a care house’, *IEEE Transactions on Robotics*, 23(5), pp. 972-980. Available at: <https://ieeexplore.ieee.org/document/4339551> (Accessed: 18 March 2026).

2- Task (4) Novel Programming Project (Adaptiveness in Assistive Robotics)

2.1. Introduction (10%; Salman)

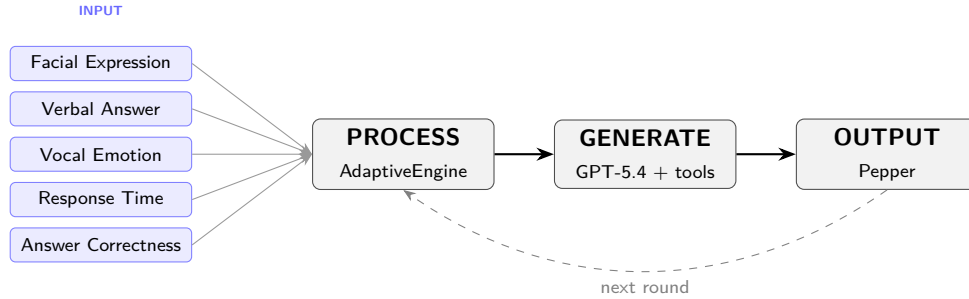


Figure 3: GAZE system architecture. Five live input signals feed the AdaptiveEngine (the symbolic reasoning layer), wherein a context block is built and passed to GPT-5.4 function-calling; a separate deterministic GPT-4.1 call handles answer verification (i.e. the referee, not the host). The generated response delivers concurrently via Pepper’s speech, gesture, and LED subsystems, whilst the dashboard pulls frames from Pepper’s ALVideoDevice socket stream rather than still photos. The loop circulates to the next round, now adapted.

As noted in {video of easy questions} wherein Salman purposely disengages from the human-robot interaction, and GAZE-Pepper

2.2. Background (10%; Alfie)

GAZE sits within socially assistive robotics (*the deployment of robots to support users through social interaction rather than physical contact*) and affective computing, which Picard (1997, p. 1, Abstract) founds as “computing that relates to, arises from, or influences emotions”; Tapus, Matarić and Scassellati (2007, .pdf-p. 1) define this sub-field as systems that “assist users through social interaction.” Most-current platforms react to a single input signal, suffering the single-signal problem: a facial-expression classifier misreads resting faces as displeasure; a response-time metric mistakes deliberation for disengagement. Calvo and D’Mello (2010, p. 28) identify “the inherent challenges with unisensory affect detection”; Poria et al. (2017, p. 99) report that multimodal systems were “consistently (85% of systems) more accurate than their best unimodal counterparts, with an average improvement of 9.83%.” Spezialetti, Placidi and Rossi (2020, pp. 1-2, Introduction) substantiate this within HRI via reviewings of recognition systems across facial, vocal, and physiological channels. This demands multi-signal fusion (the system weighs complementary channels together instead of trusting any one alone).

GAZE’s core contribution: multi-signal emotional inference; facial expression (CNN, Workshop 10), vocal emotion (MLP, Workshop 8), speech volume/RMS, response time, answer correctness, and answer text fuse with derived temporal signals into a single-inferred user-state. The implementation is face-primary: voice nudges only when the face is Neutral and the MLP clears 0.9 confidence (**fearful** excluded as the silence-attractor). GPT-5.4 generates dialogue whilst the symbolic AdaptiveEngine governs state inference,

grounding output in Pepper’s affordances (Ahn et al., 2022, *.pdf*-p. 1, Abstract); a GPT-4.1 verifier handles answer-checking. Smedegaard (2019, p. 414, Experiential novelty) warns engagement reflects novelty rather than sustained interest. GAZE’s adaptive engine therefore aims to sustain engagement beyond the novelty phase by adaptively (dynamically) adjusting difficulty, switching games, and drafting adapted questions based on inferred user-state.

2.3. Methods & Setup (35%; Alfie)

2.3.1 System Architecture

- ☒ PROOFREAD GAZE operates as a conversational loop rather than a rigid question-answer cycle. Each turn: 1) Pepper’s streamed camera frame (live `ALVideoDevice` socket, not per-turn photo) is classified for facial expression; 2) Pepper records through all four microphones with ambient-calibrated silence detection; 3) the WAV is canonicalised to mono 16 kHz by loudest-channel selection, then processed for vocal emotion, volume/RMS, and Whisper transcription; 4) the AdaptiveEngine infers user-state from face, voice, volume, response time, correctness, and answer text; 5) context plus user speech are sent to GPT-5.4 with four function-calling tools; 6) speech, gesture, and LED state fire concurrently on Pepper. Computation runs on the laptop; Pepper handles physical I/O. The dashboard hence reflects live emotional-state inference from session start.
- ☐ This function-calling architecture is neuro-symbolic: GPT-5.4 governs dialogue and decision-making, whilst the AdaptiveEngine and game logic are exposed as callable tools. This aligns with Garcez and Lamb’s (2023, *.pdf*-p. 1) ‘third wave’ paradigm, wherein neural and symbolic components share a structured interface (cf. Ahn et al., 2022, *.pdf*-p. 1).

2.3.2 Input Layer: Multimodal and Behavioural Signals

1- Facial Expression (vision-based). A pre-trained CNN (Workshop 10) classifies the user’s expression into seven categories (Angry, Disgust, Fear, Happy, Neutral, Sad, Surprise) from a 48×48 greyscale face region, building upon Ekman and Friesen (1971, p. 128), whose cross-cultural results show that “particular facial behaviors are universally associated with particular emotions,” finding that even preliterate (without written language) cultures with “minimal opportunity to have learned to recognize uniquely Western facial expressions” identified the same six emotions; this taxonomy remains “still the most popular perspective for FER” (Li and Deng, 2020, p. 1). Known limitations (cultural bias, resting-face misclassification) are mitigated by combining facial evidence with behavioural signals (response time, correctness, silence history, volume/RMS) rather than allowing the CNN to decide user state alone.

2- Verbal Answer (speech-based). Pepper records through all four microphones (the `[1,1,1,1]` mask) via `ALAudioRecorder` at 16 kHz, whilst `ALAudioDevice` polls the maximum energy across all four mics, hence catching off-axis speakers a front-only poll would miss. Recording terminates after 1.2 seconds of post-speech silence or at the adaptive hard ceiling. After SFTP, `force_mono_16k_wav()` selects the loudest channel by RMS and rewrites mono 16 kHz PCM. The canonical WAV is then transcribed via OpenAI Whisper (`whisper-1`), whose models “approach [human] accuracy and robustness” (Radford et al., 2023, Abstract). Whisper is utilised as a delivery-aware sensor: `verbose_json` exposes per-segment `no_speech_prob`, `avg_logprob`, and `compression_ratio` diagnostics; it therefore encodes *how* the user spoke alongside *what* they said, cues older ASR strips wholesale. Turn-taking is strictly sequential, wherein the mic disengages during TTS playback; barge-in (user-initiated interrupt mid-utterance) was therefore deliberately omitted, owing to NAOqi’s imperfect acoustic echo cancellation (which would have the robot’s own voice fire false interrupts) and the resultant false-trigger risk under noisy demo conditions.

Listing 1: `nao_record`: calibrated-threshold silence detection polling the max energy across all four Pepper mics, with firmware fallback. Recording uses the `[1,1,1,1]` channel mask; `force_mono_16k_wav()` downstream picks the Loudest channel by RMS so Whisper, Silero-VAD, the WS-08 MLP, and `measure_volume()` all see one canonical mono 16 kHz layout.

```

1 def nao_record(ssh, energy_threshold: int = DEFAULT_ENERGY_THRESHOLD,
2               record_max_secs: float = RECORD_MAX_SECS,
3               silence_secs: float = SILENCE_DURATION):

```

```

4      """Record audio on Pepper with dynamic silence detection.
5      - poll the max energy across all four mics to stop recording early if silence detected
6      - calibrated energy threshold to avoid false positives from ambient noise
7      - if getFrontMicEnergy is unsupported (e.g. older firmware), fall back to a safe
        ↳ fixed-duration recording to ensure the demo still works, albeit without silence
        ↳ detection
8      """
9
10     nao_run(ssh, f"""
11 from naoqi import ALProxy
12 import time
13
14 rec = ALProxy("ALAudioRecorder", "127.0.0.1", 9559)
15
16 rec.stopMicrophonesRecording()
17 rec.startMicrophonesRecording("{REMOTE_WAV}", "wav", 16000, [1, 1, 1, 1])
18
19 try:
20     audio = ALProxy("ALAudioDevice", "127.0.0.1", 9559)
21
22     speech_detected = False
23     silence_start = None
24     start = time.time()
25     threshold = {energy_threshold}
26
27     while True:
28         elapsed = time.time() - start
29
30         # hard ceiling; never exceed max duration
31         if elapsed >= {record_max_secs}:
32             break
33
34         # poll all four mics and take the MAX; a user stood to either side of Pepper registers on the
        ↳ side mics but not the front, so front-only polling misses them and the silence-detector
        ↳ stops recording mid-utterance
35     try:
36         energy = max(
37             audio.getFrontMicEnergy(),
38             audio.getLeftMicEnergy(),
39             audio.getRightMicEnergy(),
40             audio.getRearMicEnergy(),
41         )
42     except Exception:
43         energy = audio.getFrontMicEnergy() # firmware fallback
44
45     if elapsed < {RECORD_MIN_SECS}:
46         # minimum recording period
47         if energy > threshold:
48             speech_detected = True
49             time.sleep({SILENCE_POLL_SECS})
50             continue
51
52     if energy > threshold:
53         speech_detected = True
54         silence_start = None
55     else:
56         if speech_detected and silence_start is None:
57             silence_start = time.time()
58         if speech_detected and silence_start is not None:

```

```

59         if (time.time() - silence_start) >= {silence_secs}:
60             break
61
62     time.sleep({SILENCE_POLL_SECS})
63
64 except Exception as e:
65     # firmware fallback; getFrontMicEnergy() unsupported on this Pepper
66     # fall back to a safe fixed-duration recording so the demo never breaks
67     print(" [Silence detection failed: " + str(e) + "] Falling back to fixed-duration recording")
68     time.sleep({record_max_secs})
69
70 rec.stopMicrophonesRecording()
71 """
72     sftp = ssh.open_sftp()
73     sftp.get(REMOTE_WAV, LOCAL_WAV)
74     sftp.close()

```

☒ PROOFREAD Whisper hallucinates on near-silent audio, emitting training-set tics (CJK gibberish, prompt-primed repetition); the latter is NLG *degeneration* wherein models get “stuck in repetitive loops” (Ji et al., 2023, p. 3). Transcription is therefore gated by a five-layer defence chain (Listing~2); the Vosk wake-word gate is retained but currently bypassed pipeline-wide once Silero, Whisper-self-signals, and the blacklist proved sufficient.

Listing 2: `transcribe`: five-layer defence chain against Whisper hallucination on near-silent audio (`gaze.py`, lines 1532–1610).

```

1  def transcribe(bypass_wake_word: bool = False,
2                 whisper_prompt: str = "User answers a quiz or chats with a companion robot.",
3                 record_again=None,
4                 max_hallucination_retries=None) -> str:
5     # Silero VAD -> wake-word -> Whisper -> hallucination blacklist
6     attempts_remaining = max_hallucination_retries
7     while True:
8         if INPUT_IS_LOCAL and not _local_speech_detected:
9             return ""
10
11         # Silero VAD hard gate; NAO + LOCAL
12         if not has_real_speech(LOCAL_WAV):
13             print(" Silero VAD found no speech; skipping Whisper.")
14             return ""
15
16         # Vosk wake-word gate; bypass for name prompt
17         if not bypass_wake_word and not has_wake_word(LOCAL_WAV):
18             print(" No wake-word detected; skipping Whisper.")
19             return ""
20
21     try:
22         with open(LOCAL_WAV, "rb") as fh:
23             resp = client.audio.transcriptions.create(
24                 model="whisper-1", #
25                 file=fh, #
26                 response_format="verbose_json", # get self-signals for hallucination detection
27                 temperature=0.0, # zero randomness
28                 prompt=whisper_prompt,
29                 timeout=API_TIMEOUT,
30             )
31         text = (getattr(resp, "text", "") or "").strip()
32         print(f" Whisper raw text: {text!r}")
33

```

```

34     # Whisper self-signals from verbose_json
35     segments = getattr(resp, "segments", None) or []
36     suspected_hallucination = False
37     if segments:
38         no_speech_vals = [s.no_speech_prob for s in segments
39                           if getattr(s, "no_speech_prob", None) is not None]
40         logprob_vals = [s.avg_logprob for s in segments
41                         if getattr(s, "avg_logprob", None) is not None]
42         compression_vals = [s.compression_ratio for s in segments
43                             if getattr(s, "compression_ratio", None) is not None]
44         print(f" Whisper diagnostics: no_speech={no_speech_vals},
45               ↪ avg_logprob={logprob_vals}, compression={compression_vals}")
46         if no_speech_vals and max(no_speech_vals) > 0.6:
47             print(f" Whisper flagged silence (max no_speech_prob={max(no_speech_vals):.2f});
48                   ↪ dropping {text!r}")
49             return ""
50         if logprob_vals and min(logprob_vals) < -1.3:
51             print(f" Whisper low-confidence (min avg_logprob={min(logprob_vals):.2f});
52                   ↪ dropping {text!r}")
53             return ""
54         # repetition-loop hallucination; flag for retry path
55         if compression_vals and max(compression_vals) > 2.4:
56             print(f" Whisper repetition loop (max
57                   ↪ compression_ratio={max(compression_vals):.2f}); flagging {text!r} as
58                   ↪ hallucination")
59             suspected_hallucination = True
60
61     # normalised hallucination blacklist + Whisper-self-signal hallucinations
62     if suspected_hallucination or is_known_hallucination(text):
63         print(f" Filtered Whisper hallucination: {text!r}")
64         if record_again is not None and (attempts_remaining is None or attempts_remaining >
65                                         ↪ 0):
66             if attempts_remaining is not None: # whilst more retries remain
67                 attempts_remaining -= 1
68                 print(f" Disregarding hallucination; listening again (attempts left:
69                       ↪ {attempts_remaining}).")
70             else:
71                 print(f" Disregarding hallucination; listening again.")
72                 record_again()
73                 continue
74     return ""
75
76     # Strip leading "Pepper"/"Gaze" so handlers receive just the answer; \b blocks
77     ↪ "Pepperoni"/"Gazebo" false positives..
78     stripped = re.sub(r'(?i)^\s*(pepper|gaze)\b[.,\s]*', '', text).strip()
79     return stripped
80 except Exception as e:
81     print(f" Whisper transcribe failed ({e}); returning empty")
82     return ""

```

3- Vocal Emotion (audio-based). The same WAV is passed through a pre-trained MLP (Workshop 8) before transcription, classifying vocal state into four emotions (calm, happy, fearful, disgust) via MFCC, chroma, and mel-spectrogram features; El Ayadi, Kamel and Karray (2011, p. 578) identify MFCCs as “the most promising features” for speech-emotion recognition. This provides a second, independent modality; the two may disagree, wherein decision logic arbitrates. RAVDESS is an acted-speech corpus, and indeed Williams and Stevens (cited in El Ayadi et al., 2011, p. 573) found “acted emotions tend to be more exaggerated than real ones”. The WS-08 feature pipeline mean-pools MFCC, chroma, and mel across the utterance into a single global vector (*i.e. one fixed-size summary per clip*), wherein “temporal information present in

speech signals is completely lost” (ibid., p. 574); fear’s defining tremor and irregular-pitch structure consequently collapses, indistinguishable from low-energy real-room speech. Voice is therefore engineered as a tie-breaker (the gate is detailed in Section 2.3.3), an architectural response to a documented limitation rather than a debugging afterthought.

Listing 3: `SpeechEmotionModel.extract_features`: MFCC + chroma + mel-spectrogram feature vector matching the WS-08 training pipeline, with peak-normalisation for quieter brain-injured users (gaze.py, lines 264–292).

```

1
2 @staticmethod # static because it's also used independently in classify_speech_emotion()
3     def extract_features(wav_path: str):
4         "Extract the same MFCC/chroma/mel feature vector used in WS-08 training."
5         with sf.SoundFile(wav_path) as sound_file:
6             audio = sound_file.read(dtype="float32")
7             sample_rate = sound_file.samplerate
8
9         if audio.ndim > 1:
10             audio = audio.mean(axis=1)
11
12         # peak-normalise; helps quieter speakers
13         audio = librosa.util.normalize(audio)
14
15         n_fft = 2048
16         if len(audio) < n_fft:
17             return None
18
19         stft = np.abs(librosa.stft(audio, n_fft=n_fft))
20
21         mfccs = np.mean(librosa.feature.mfcc(
22             y=audio, sr=sample_rate, n_mfcc=40).T, axis=0).flatten()
23
24         chroma = np.mean(librosa.feature.chroma_stft(
25             S=stft, sr=sample_rate).T, axis=0).flatten()
26
27         mel = np.mean(librosa.feature.melspectrogram(
28             y=audio, sr=sample_rate).T, axis=0).flatten()
29
30         return np.concatenate([mfccs, chroma, mel])

```

4- Response Time (engagement-based). A Python timer measures elapsed time from question delivery to recording completion; the Whisper call occurs *after* the timer halts, isolating deliberation time from API latency.

5- Answer Correctness (task-based). GPT-4.1’s `check_game_answer` evaluates the transcribed answer at a deterministic temperature; the resultant binary correctness feeds the AdaptiveEngine and the derived rolling-correctness signal.

2.3.3 Process Layer: Multi-Signal State Inference

The AdaptiveEngine’s `infer_state()` combines multimodal and behavioural signals to classify the user into one of five states: *Thriving*, *Comfortable*, *Struggling*, *Frustrated*, or *Disengaged*. Five main input channels are supplemented by three derived temporal features: rolling correctness over the last five rounds, consecutive-silence count, and consecutive-wrong streak length. Crucially, classification is face-primary: facial expression, correctness, response time, silence, wrong-streaks, and volume carry the decision; vocal emotion fires only as a high-confidence tie-breaker when face is Neutral AND the MLP clears 0.9 AND the label is not **fearful** (the silence-attractor; RAVDESS’s actor-exaggerated fear collides with low-energy real-room speech in the mean-pooled feature-space, hence the gate). Voice is thus retained as evidence but never allowed to override

stronger behavioural readings, addressing the brittleness Desai et al. (2013) observe in single-signal systems. Thresholds (correctness floor 0.4, ceiling 0.8, response-time baseline 15s, consecutive-wrong trigger 3, silence threshold 2) were derived from pilot testing.

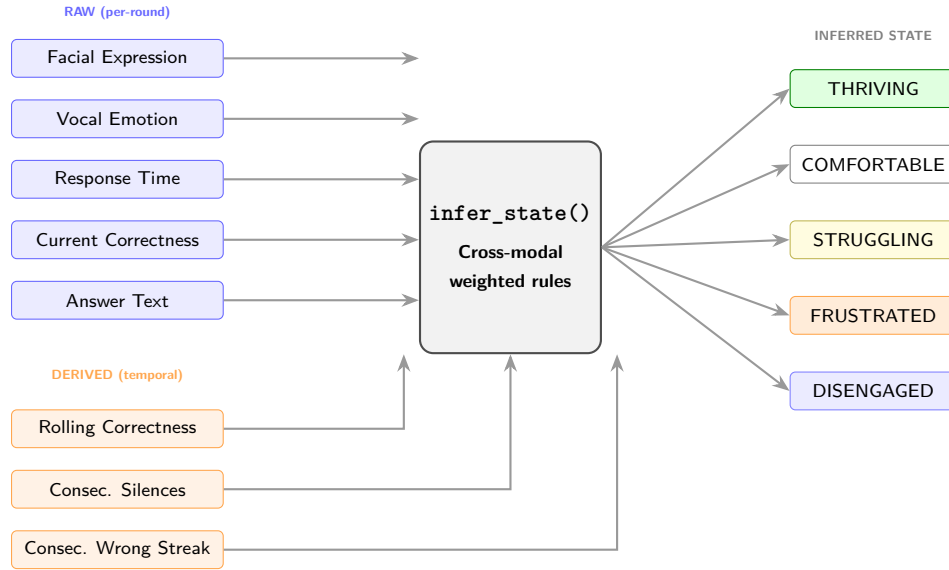


Figure 4: Multi-signal inference pipeline. Five raw inputs captured each round and three derived temporal signals computed from session history feed into `infer_state()`, which applies cross-modal rules (e.g. camera reads *Angry* but user answers fast and correctly $\rightarrow$ *Comfortable*) to classify the user into one of five states. Voice is treated as advisory rather than dominant: it nudges the state only when the face is Neutral AND the MLP’s confidence clears 0.9 AND the label is not *fearful* (the silence-attractor), hence a noisy vocal label can never override stronger visual or behavioural evidence; therein lies the multi-signal novelty.

The complete classification logic, face-primary with voice tie-breakers, is shown in Listing~4 (extracted from `gaze.py infer_state()`):

Listing 4: Multi-signal inference rules (verbatim excerpt from `gaze.py infer_state()`; method-body indentation stripped). The newest implementation is face-primary: every rule fires off facial expression + behavioural signals first, voice is consulted only as a high-confidence tie-breaker at the end.

```

1
2 def infer_state(self, expression: str, response_time: float,
3                 correct: bool, answer_text: str,
4                 vocal_emotion: str = "neutral",
5                 vocal_conf: float = 0.0,
6                 volume_rms: float = 0.0) -> InferredState:
7     """
8     Infer the user's state from all signals.
9     Face is primary; voice is only derived when face is Neutral and
10    the voice signal is high-confidence and not "fearful" (the MLP
11    collapses to "fearful" in silence).
12    """
13    correctness = self.rolling_correctness()
14    clean = answer_text.strip().lower()
15    is_silent = (not clean or clean in {"i don't know", "skip", "pass", "next"}) # if no
        ↪ meaningful input || input matches a skip-command phrase in the set (set over list
        ↪ because it's faster) then indeed silent (True)
16
17    # Arousal bounds calibrated against ambient noise

```

```

18     high_arousal = volume_rms > self.VOLUME_LOUD
19     low_arousal = 0 < volume_rms < self.VOLUME_QUIET
20
21     # voice trusted only when not-fearful
22     trust_voice = (vocal_conf >= 0.9 and vocal_emotion != "fearful")
23
24     if is_silent:
25         self.consecutive_silences += 1
26     else:
27         self.consecutive_silences = 0
28     if correct:
29         self.consecutive_correct += 1
30         self.consecutive_wrong = 0
31     else:
32         self.consecutive_wrong += 1
33         self.consecutive_correct = 0
34
35     # 1- FACE-PRIMARY RULES: these fire before voice is ever consulted
36
37     # thriving: good performance + fast responses
38     if (correctness >= CORRECTNESS_CEILING and response_time < RESPONSE_TIME_BASELINE * 0.5):
39         return InferredState.THRIVING
40     if expression == "Angry" and correct and response_time < RESPONSE_TIME_BASELINE * 0.6:
41         return InferredState.COMFORTABLE
42
43     # disengaged: silence + slow + poor performance
44     if self.consecutive_silences >= SILENCE_THRESHOLD:
45         return InferredState.DISENGAGED
46     if (expression == "Neutral"
47         and response_time > RESPONSE_TIME_BASELINE
48         and correctness < 0.5):
49         return InferredState.DISENGAGED
50     if (low_arousal and expression == "Neutral"
51         and response_time > RESPONSE_TIME_BASELINE * 0.8):
52         return InferredState.DISENGAGED
53
54     # frustrated: negative face + poor performance
55     if expression in ("Angry", "Disgust") and correctness < CORRECTNESS_FLOOR:
56         return InferredState.FRUSTRATED
57     if self.consecutive_wrong >= 3 and expression in ("Angry", "Sad", "Fear"):
58         return InferredState.FRUSTRATED
59     if (high_arousal and expression in ("Angry", "Disgust", "Fear")
60         and correctness < CORRECTNESS_FLOOR):
61         return InferredState.FRUSTRATED
62
63     # struggling: sadness + slow; poor correctness; fear + wrong
64     if expression == "Sad" and response_time > RESPONSE_TIME_BASELINE * 0.7:
65         return InferredState.STRUGGLING
66     if correctness < CORRECTNESS_FLOOR:
67         return InferredState.STRUGGLING
68     if expression == "Fear" and not correct:
69         return InferredState.STRUGGLING
70
71     # 2- voice tie-breakers; only when face neutral
72     if expression == "Neutral" and trust_voice:
73         if vocal_emotion == "happy" and correct and correctness >= CORRECTNESS_CEILING:
74             return InferredState.THRIVING
75         if vocal_emotion == "calm" and correctness >= 0.5:
76             return InferredState.COMFORTABLE

```

77
78

```
return InferredState.COMFORTABLE # default: face gave no negative signal, performance is
    ↪ holding
```

The `decide()` function then maps the inferred state to concrete adaptive actions: difficulty adjustment (easy, medium, hard), game switching (numbers ↔ letters when the user is frustrated or disengaged), hint provision, encouragement, and tone selection. This parallels Nikolaidis, Hsu and Srinivasa’s (2017, p. 625) mutual-adaptation paradigm. Table~1 summarises the mapping.

Inferred State	Difficulty	Game Switch	Hint	Tone	LED Colour
Thriving	↑ ramp up	No	No	Energetic	Green
Comfortable	↑ if >70%	No	No	Neutral	Cyan
Struggling	↓ ease off	No	Yes	Encouraging	Yellow
Frustrated	↓↓ Easy	After 4 wrong	No	Calm	Red
Disengaged	—	After 3 checkouts	No	Energetic	Magenta

Table 1: State-to-action mapping. The adaptive engine translates each inferred state into a specific combination of difficulty adjustment, game-type switch, hint provision, tone selection, and LED colour. Arrows indicate direction of difficulty change relative to the current level, wherein the mapping applies solely to `decide()`’s engine output (the spoken-checkout path, e.g. repeated *skip*); the main-loop tiered intervention on pure-silence turns, tier-1 check-in at 3 silences and tier-2 switch at 4, bypasses `decide()` entirely.

Disengagement is flagged by three OR’d rules wherein any one trips it: 1) two consecutive silences (mic-empty or “skip”/“pass” checkout phrases), 2) neutral-face + response > 15s + rolling-correctness < 50% (the cognitive-checkout pattern), 3) low-volume + neutral-face + slow-ish response (early-warning drift, catches it before accuracy tanks). GAZE then tiers its intervention, gentle check-in at three silences, game-switch at four, wherein each tier fires at most once per silent spell.

Complementing state inference, `recommend_think_budget()` combines five signals: inferred state, facial expression, prior response-time, `consecutive_silences`, and the LLM’s `waiting` flag. Crucially, `waiting` is treated as one signal among many rather than the sole path; `request_more_time` extends additively whilst the four behavioural signals extend independently, addressing the brittleness Desai et al. (2013) observe in single-signal systems. Round 1’s generous default (7s/2.5s/15s) is target-user-aware (aphasia, arithmetic-pause tolerance for stroke-recovery users); a defensive 20s ceiling caps `record_max_secs` under `CMD_TIMEOUT` against signal-compound edge cases.

2.3.4 Generate Layer: Dynamic Prompt Construction

☒ PROOFREAD

- ☐ Rather than constructing a fixed game prompt each round, GAZE utilises OpenAI function calling to let GPT-5.4 decide actions. Every turn, `build_signal_context()` packages live signals (face, voice, volume, response time, rolling accuracy, recent-face/recent-vocal windows) into a context block prepended to the user’s transcribed speech. This is sent to GPT-5.4 with four callable tools: `generate_game_question`, `check_game_answer`, `evaluate_last_adaptation`, and `request_more_time`. The LLM decides which to invoke; during natural conversation it calls none, whilst during gameplay it sequences `check_game_answer` and `generate_game_question`. Game-question generation runs as a separate JSON-only call with a variety seed plus a rolling 30-question do-not-repeat memory, hence preventing mode-collapsed targets. Answer verification is delegated to a deterministic GPT-4.1 verifier (temperature 0.0), reducing the hallucination risk Ji et al. (2023, p. 3) identify. A 10-second timeout wraps every API call in case the network stalls so Pepper falls back gracefully rather than freezing mid-turn.

A second hallucination safeguard inside `build_signal_context()` *semantically buckets* signals (e.g. `System` ↪ `pacing: relaxed and patient` rather than raw `Think budget: 18s`), preventing the dialogue temperature

(0.8) from echoing integers verbatim; a failure mode Ji et al. (2023, p. 3) classify under NLG hallucination. The answer-checking path eliminates hallucination via determinism, whilst the dialogue path eliminates it via signal abstraction.

2.3.5 Output Layer: Aligned Multimodal Response

Speech, gestures, and LED state fire concurrently via threading. Context-aligned gestures (e.g. animated speech for celebratory moments) execute alongside dialogue, whilst LED colours reflect the inferred state, providing a secondary non-verbal feedback channel.

2.3.6 Adaptation Self-Evaluation and Session Persistence

- ☒ PROOFREAD GAZE exposes `evaluate_adaptation()` to the LLM as the `evaluate_last_adaptation` tool, hence the LLM can compare consecutive rounds to judge whether a previous adaptation helped (e.g. did a difficulty decrease after frustration produce a correct answer?). Session progress is saved to `gaze_save.json` after every game round, with a belt-and-braces auto-save every two turns regardless, in case the session is interrupted before the next scheduled save.

2.4. Outcome & System Analysis (30%; Salman)

2.5. Conclusion (10%; Alfie)

GAZE implements multi-signal emotional inference across two independent modalities (*facial expression via WS-10 CNN and vocal emotion via WS-08 MLP*) alongside speech volume/RMS, response time, answer correctness, answer text, and derived temporal signals, hence yielding a more robust user-state estimate than any single channel. The implementation hardens further by recording all four Pepper microphones, canonicalising audio to mono 16-kHz by loudest-channel selection, applying Silero-VAD to both modes, and streaming Pepper’s camera via persistent `ALVideoDevice` rather than per-turn SFTP. The hybrid architecture (GPT-5.4 dialogue paired with symbolic rule-based inference and sometimes deterministic GPT-4.1 verifiers); adaptive game-switching directly target the novelty-decay problem Smedegaard (2019, p. 414, Experiential novelty) identifies.

- ☒ PROOFREAD The conversational architecture, wherein the LLM decides actions via function calling, positions GAZE as a social companion rather than a rigid game host. Every turn, the AdaptiveEngine’s `decide()` resolves a tone label (encouraging, celebratory, calm, energetic, neutral) from the inferred state; the LLM’s dialogue register thereby modulates live rather than holding a fixed persona, extending Tapus, Tapus and Mataric’s (2008) personality-matching paradigm from static trait-fit to dynamic state-driven adaptation.

The multi-signal approach could transfer to stroke rehabilitation re-engagement (Mataric et al., 2007), educational tutoring, or neurodivergent support. Future work could replace hand-coded thresholds with learned parameters from longitudinal data, and fine-tune both models on in-session Pepper captures. GAZE sits within Section 1’s cognitive trajectory rather than the reactive social layer alone.

2.6 Task-4 References (5%)

- ☒ verify all references
- Ahn, M., Brohan, A., Brown, N., et al. (2022) ‘Do As I Can, Not As I Say: Grounding Language in Robotic Affordances’, *arXiv preprint arXiv:2204.01691*. Available at: <https://arxiv.org/abs/2204.01691> (Accessed: 24 March 2026).
- Desai, M., Kaniarasu, P., Medvedev, M., Steinfeld, A. and Yanco, H. (2013) ‘Impact of robot failures and feedback on real-time trust’, *Journal of Human-Robot Interaction*, 2(1), pp. 251–275. Available at: <https://ieeexplore.ieee.org/document/6483596> (Accessed: 20 March 2026).

- Fong, T., Nourbakhsh, I. and Dautenhahn, K. (2003) ‘A survey of socially interactive robots’, *Robotics and Autonomous Systems*, 42(3–4), pp. 143–166. Available at: <https://www.cs.cmu.edu/~illah/PAPERS/socialroboticssurvey.pdf> (Accessed: 18 March 2026).
- Garcez, A. d’A. and Lamb, L. C. (2023) ‘Neurosymbolic AI: The 3rd Wave’, *Artificial Intelligence Review*, 56, pp. 12387–12406. Available at: <https://link.springer.com/article/10.1007/s10462-023-10448-w> (Accessed: 20 March 2026).
- Ji, Z., Lee, N., Frieske, R., et al. (2023) ‘Survey of Hallucination in Natural Language Generation’, *ACM Computing Surveys*, 55(12), pp. 1–38. Available at: <https://dl.acm.org/doi/10.1145/3571730> (Accessed: 22 March 2026).
- Kaelbling, L. P., Littman, M. L. and Cassandra, A. R. (1998) ‘Planning and acting in partially observable stochastic domains’, *Artificial Intelligence*, 101(1–2), pp. 99–134. Available at: <https://people.csail.mit.edu/lpk/papers/aij98-pomdp.pdf> (Accessed: 13 March 2026).
- Nikolaidis, S., Hsu, D. and Srinivasa, S. (2017) ‘Human-robot mutual adaptation in collaborative tasks: Models and experiments’, *The International Journal of Robotics Research*, 36(5–7), pp. 618–634. Available at: <https://journals.sagepub.com/doi/10.1177/0278364917690593> (Accessed: 20 March 2026).
- Picard, R.W. (1997) ‘Affective Computing’, *MIT Media Laboratory Perceptual Computing Section Technical Report No. 321*. Available at: <https://affect.media.mit.edu/pdfs/95.picard.pdf> (Accessed: 14 April 2026).
- Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C. and Sutskever, I. (2023) ‘Robust Speech Recognition via Large-Scale Weak Supervision’, in *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, PMLR 202, pp. 28492–28518. Available at: <https://proceedings.mlr.press/v202/radford23a/radford23a.pdf> *Proceedings of Machine Learning Research* (Accessed: 14 April 2026).
- Sciutti, A., Beetz, M., Inamura, T., et al. (2023) ‘The Present and the Future of Cognitive Robotics’, *IEEE Robotics & Automation Magazine*, 30(3), pp. 160–163. Available at: <https://ieeexplore-ieee.org/plymouth.idm.oclc.org/document/10255092> (Accessed: 18 March 2026).
- Smedegaard, C. V. (2019) ‘Reframing the Role of Novelty within Social HRI: From Noise to Information’, in *Proceedings of the 14th ACM/IEEE International Conference on Human-Robot Interaction (HRI ’19)*, pp. 411–420. Available at: <https://dl.acm.org/doi/10.1109/HRI.2019.8673219> (Accessed: 22 March 2026).
- Spezialetti, M., Placidi, G. and Rossi, S. (2020) ‘Emotion Recognition for Human-Robot Interaction: Recent Advances and Future Perspectives’, *Frontiers in Robotics and AI*, 7, Article 532279. Available at: https://www.researchgate.net/publication/347520928_Emotion_Recognition_for_Human-Robot_Interaction_Recent_Advances_and_Future_Perspectives (Accessed: 14 April 2026).
- Tapus, A., Matarić, M. J. and Scassellati, B. (2007) ‘Socially assistive robotics [Grand Challenges of Robotics]’, *IEEE Robotics & Automation Magazine*, 14(1), pp. 35–42. Available at: <https://sczlab.yale.edu/sites/default/files/files/Tapus-RAM2007.pdf> (Accessed: 25 March 2026).

3. Appendix

3.1 Video Demo

- YouTube link: [test](#)

3.2 AI Declaration

Solo Work	S1 - Generative AI tools have not been used for this assessment.
Assisted Work	A1 – Idea Generation and Problem Exploration Used to generate project ideas, explore different approaches to solving a problem, or suggest features for software or systems. Students must critically assess AI-generated suggestions and ensure their own intellectual contributions are central.
	A2 - Planning & Structuring Projects AI may help outline the structure of reports, documentation and projects. The final structure and implementation must be the student's own work.
	A3 – Code Architecture AI tools maybe used to help outline code architecture (e.g. suggesting class hierarchies or module breakdowns). The final code structure must be the student's own work.
	A4 – Research Assistance Used to locate and summarise relevant articles, academic papers, technical documentation, or online resources (e.g. Stack Overflow, GitHub discussions). The interpretation and integration of research into the assignment remain the student's responsibility.
	A5 - Language Refinement Used to check grammar, refine language, improve sentence structure in documentation not code. AI should be used only to provide suggestions for improvement. Students must ensure that the documentation accurately reflects the code and is technically correct.
	A6 – Code Review AI tools can be used to check comments within the code and to suggest improvements to code readability, structure or syntax. AI should be used only to provide suggestions for improvement. Students must ensure that the code accurately reflects their knowledge and is technically correct.
	A7 - Code Generation for Learning Purposes Used to generate example code snippets to understand syntax, explore alternative implementations, or learn new programming paradigms. Students must not submit AI-generated code as their own and must be able to explain how it works.
	A8 - Technical Guidance & Debugging Support AI tools can be used to explain algorithms, programming concepts, or debugging strategies. Students may also help interpret error messages or suggest possible fixes. However, students must write, test, and debug their own code independently and understand all solutions submitted.
	A9 - Testing and Validation Support AI may assist in generating test cases, validating outputs, or suggesting edge cases for software testing. Students are responsible for designing comprehensive test plans and interpreting test results.
	A10 - Data Analysis and Visualization Guidance AI tools can help suggest ways to analyse datasets or visualize results (e.g. recommending chart types or statistical methods). Students must perform the analysis themselves and understand the implications of the results.
	A11 - Other uses not listed above Please specify:
Partnered Work	P1 - Generative AI tool usage has been used integrally for this assessment Students can adopt approaches that are compliant with instructions in the assessment brief. Please Specify:

Figure 5: Student Declaration of AI Tool use in this Assessment Table

I declare that I've used the AI tools listed below whilst preparing this assessment. I've read and understood the University of Plymouth's policy on the use of AI tools in assessment and confirm that my use falls within

the coursework's allowed categories, i.e. **A2 (Planning and Structuring Projects)** and **A4 (Research Assistance)**.

AI Tool Used	Purpose of Use	Extent of Use
ChatGPT	Brainstorming project ideas and structuring the report (A2)	Initial brainstorming and final outline stages for GAZE
ChatGPT	Reviewing structural alignment against grading criteria and mapping word-count budgets (A2)	After and midway through section-drafting
ChatGPT	Finding relevant pages to read in the paper (A4)	Few times if the paper is too long
ChatGPT	General conversations via web-search AI about prevalent papers to read about how the topics relates to others' studies (A4)	Few times at the end

- ☒ I understand that the ownership and responsibility for the academic integrity of this submitted assessment falls with me, the student.
- ☒ I confirm that all details provided above are an accurate description of how AI was used for this assessment.

3.3 gaze22.py

```

1  """
2  GAZE: Game-Adaptive Zone of Engagement.
3
4  A Pepper (NAO) countdown-game host. The robot adapts difficulty,
5  pacing and feedback per turn from five signals (each paired with the function wherein it is
    ↳ produced):
6      1- facial expression (CNN, WS-10)    primary    --->    capture_and_classify() /
    ↳ FacialExpressionModel.predict()
7      2- answer correctness (performance) primary    --->    check_answer() (GPT-4.1 verifier)
8      3- response time (behavioural)       primary    --->    time.time() - question_start (main
    ↳ loop)
9      4- speech volume (RMS)                secondary --->    local_record() / nao_record() (RMS per
    ↳ audio chunk)
10     5- vocal emotion (MLP, WS-08)          tie-breaker due to difficulty with model selection --->
    ↳ classify_speech_emotion() / SpeechEmotionModel.predict()
11     All five fan in to: AdaptiveEngine.infer_state() -> AdaptiveEngine.decide()
12
13  NOVELTY:
14  Multi-signal integration: single signals are not trusted alone. Voice is only consulted when the
15  face is Neutral, vocal-emotion confidence is >= 0.9 and the label is not "fearful"; the MLP might
16  collapse to it on quiet/noisy audio.
17
18  ARCHITECTURE: gpt-5.4 converse with access to four function-calling tools:
19  - def generate_game_question (generate values computationally); def check_game_answer
20  - def evaluate_last_adaptation; def request_more_time).
21
22  Initially used a wake-word defence but became unnecessary
23
24  `AdaptiveEngine`: five signals
25  and adapts difficulty, pacing/think-budget, and game-switching. A
26  deterministic GPT-4.1 verifier handles answer-checking.
27
28  VARIOUS MODES:
29      1- LOCAL_MODE=true:
30          - webcam; works with Mac and Windows (NAO's direct camera )
31          - Mac/Windows mic

```

```

32     - local TTS; just run entirely without robot-connection because all input/output is local
      ↪ to tester's computer
33 2- LOCAL_MODE=false:
34     - Pepper over SSH;
35     - output (TTS, LEDs, gestures) on robot.
36
37  NAO-ran code must
38
39  Now, everything is ran hybridly: dashboard stats, and the camera feed is streamed over TCP to
      ↪ the computer for
40  display and facial-expression inference. Pepper's camera is too slow for a responsive
      ↪ experience, and local webcam is much smoother. The microphone path is governed by
      ↪ LOCAL_MODE, but can be forced to the local machine with HYBRID_LOCAL_INPUT so the
      ↪ operator can speak from the computer whilst Pepper still does the talking.
41
42 3- HYBRID_LOCAL_INPUT (default = True):
43     - mic input is forced to the Mac despite LOCAL_MODE's preference
44     - essentially the brain works locally to
45
46 TESTING OBSERVATIONS:
47     - Pepper's audio trainingset is *noisy*; due to training on podcasts, youtube Whisper is
      ↪ therefore gated by defences as follows:
48     - Silero VAD
49     - no_speech_prob, and a hallucination blacklist.
50     - Vosk wake-word ensure (has_wake_word()) was indeed wired early-in to
51     combat Whisper hallucinations on noisy Pepper audio. The
52     hybrid-required switch fixed this; local Mac audio sometimes
53     produces these, whereas Pepper's stream more-often did, hence
54     wake-word is now not necessary (transcribe(bypass_wake_word=True)).
55
56     - Pepper camera streaming (~10 fps) < local webcam
57     (~30 fps); the code therefore runs hybridly: length-prefixed RGB
58     over TCP port 9558 via pepper_video_loop/pepper_camera_receive_loop;
59     detect_thread_loop runs ~6-7 Hz; display repaints ~30 fps from
60     _preview_frame+_last_bbox.
61     - The emotion classifiers are lightweight, not clinical instruments and thus
62     robustness felt unfeasible
63
64
65 PER-PROPOSAL AUTHORSHIP:
66     Alfie: architecture, OpenAI integration, AdaptiveEngine. facial-expression pipeline,
67     Salman - game flow, gestures, LEDs, TTS pacing, save/load, testing.
68
69 CRITICAL:
70 - **PROPOSAL.PDF IS SOURCE OF TRUTH FOR THE INITIAL-INTENDED DESIGN**
71 - **CONFIG NAO IP INTO ENV LIKE LAST TIME**
72
73
74 LIVE DEMO-PREP CHECKLIST:
75 - [X] fix dashboard as it is just black when NAO
76 - [X] [X] eye colours should change
77 - [X] disregard all hallucinations until sufficient listened sentence
78 - [X] fix year (escape character hallucinations)
79 - [X] offload simpler tasks? to either computation or mini model
80 - [X] ensure facial expression inference 'logics commented sufficiently
81 - [X] ensure it notices and mitigates when user's disengaged
82 - [X] fix years being hallucinated
83 - [X] fix continuation from previous game 5 games to 3 make it says results every 3 rounds for
      ↪ exampe 2 out of 3 is correct and then adapts based in that

```

```

84
85 - [X] pirotise face over voice in mulit-singal inference
86 - [X] ensure it saves all the time not just at least 5
87 times
88
89 - [X] refine the arm movement its too jiterrry
90 - [X] after escalate directly address user's disengagent
91 - [X] make it more random for more games as currently it keeps giving mainly the same answers even
    ↳ despite how hard it it is
92 - [X] make voice very not important
93 - [X] make it ask for name straight away
94
95 - [X] configure audio frequency to work better maybe for the nao (this was later resolved by
    ↳ hybrid-local-input switch to use local mic
96
97 - [X] make dashboard camera FPS much more fluid because its too slow now; fix was to make it
    ↳ stream via TCP to computer
98
99 """
100
101 import os, re, sys, json, time, types, wave, struct, socket, textwrap, tempfile, threading,
    ↳ subprocess, unicodedata
102 import tkinter as tk
103 from dataclasses import dataclass, field
104 from enum import Enum
105 from typing import Optional
106 print("[booting...] stdlib loaded", flush=True)
107
108 import numpy as np
109 import cv2
110 from PIL import Image, ImageTk
111 print("[booting...] numpy + opencv loaded", flush=True)
112
113 import paramiko
114 print("[booting...] paramiko loaded", flush=True)
115
116 import sounddevice as sd
117 import librosa
118 import soundfile as sf
119 # Vosk wake-word gate
120 try:
121     from vosk import Model as VoskModel, KaldiRecognizer
122     _vosk_import_ok = True
123 except ImportError:
124     VoskModel = None
125     KaldiRecognizer = None
126     _vosk_import_ok = False
127 # Silero VAD; pre-Whisper speech gate
128 try:
129     from silero_vad import load_silero_vad, read_audio, get_speech_timestamps
130     _silero_import_ok = True
131 except ImportError:
132     load_silero_vad = None
133     read_audio = None
134     get_speech_timestamps = None
135     _silero_import_ok = False
136 print("[booting...] audio stack loaded", flush=True)
137
138 from dotenv import load_dotenv

```

```

139 from openai import OpenAI
140 print("[booting...] openai loaded", flush=True)
141
142 import joblib
143 print("[booting...] loading tensorflow (this is slow on first run)...", flush=True)
144 import tensorflow as tf
145 from tensorflow.keras.models import model_from_json
146 print("[booting...] tensorflow loaded", flush=True)
147
148 # -----
149     ↪ -----
150 # Silero VAD loader; optional dep
151 _silero_model = None
152 if _silero_import_ok:
153     try:
154         _silero_model = load_silero_vad()
155         print("[booting] silero-vad loaded", flush=True)
156     except Exception as _vad_err:
157         print(f"[booting] silero-vad failed to load ({_vad_err}); VAD gate disabled", flush=True)
158
159 load_dotenv()
160
161
162 # NAO CONFIG
163 NAO_IP = os.getenv("NAO_IP", "ROBOT_IP")
164 NAO_USER = "nao"
165 NAO_PASS = "nao"
166 RECORD_MAX_SECS = 8 # ceiling (don't record longer than this)
167 RECORD_MIN_SECS = 1 # minimum recording before silence-detection
168 SILENCE_POLL_SECS = 0.25 # polling interval for silence detection on Pepper
169 SILENCE_DURATION = 1.2 # seconds of silence after speech to trigger stop
170 CALIBRATION_SECS = 3 # duration of start-up ambient noise calibration
171 ENERGY_BUFFER = 80 # margin above ambient baseline to set speech threshold
172 DEFAULT_ENERGY_THRESHOLD = 700 # fallback for if calibration fails
173 REMOTE_WAV = "/var/persistent/home/nao/input.wav"
174 REMOTE_IMG = "/var/persistent/home/nao/capture.jpg"
175 LOCAL_WAV = os.path.join(tempfile.gettempdir(), "gaze_input.wav")
176 LOCAL_IMG = os.path.join(tempfile.gettempdir(), "gaze_capture.jpg")
177 VOLUME_THRESHOLD = 700 # RMS; dashboard label only ("quiet" vs "normal"). Not a transcription
178     ↪ gate; see NAO_MIN_RMS_TO_TRANSCRIBE.
179 NAO_MIN_RMS_TO_TRANSCRIBE = 500 # near-empty WAV floor for the NAO-mode pre-transcribe gate;
180     ↪ Silero + Whisper + blacklist do the real filtering downstream.
181 FACE_CONFIDENCE_THRESHOLD = 0.5 # below: face uncertain, treat neutral
182 VOICE_CONFIDENCE_THRESHOLD = 0.5 # threshold for vocal emotion is too uncertain; treat as neutral
183 SSH_TIMEOUT = 10
184 CMD_TIMEOUT = 60
185
186
187 # false when connected to pepper; true for testing when no Pepper's camera
188 USE_LOCAL_CAMERA = os.getenv("GAZE_LOCAL_CAMERA", "false").lower() == "true"
189
190
191 # uses local webcam; Mac microphone, and macOS TTS instead of Pepper-hardware
192 LOCAL_MODE = os.getenv("GAZE_LOCAL_MODE", "false").lower() == "true"
193 if LOCAL_MODE:
194     USE_LOCAL_CAMERA = True
195
196
197 # Hybrid: TTS, LEDs and gestures stay on Pepper (governed by LOCAL_MODE),
198 # but the microphone path is forced to the Mac so the operator can speak from
199 # the computer whilst Pepper still does the talking. Set HYBRID_LOCAL_INPUT

```

```

195 # False to fall back to whatever LOCAL_MODE prescribes for input
196 HYBRID_LOCAL_INPUT = True
197 INPUT_IS_LOCAL = LOCAL_MODE or HYBRID_LOCAL_INPUT
198
199 DEBUG_PREVIEW = LOCAL_MODE or HYBRID_LOCAL_INPUT
200 _last_rms = 0.0 # shared with recording thread for overlay
201 _last_emotion = "" # updated by capture_and_classify
202
203 _preview_lock = threading.Lock()
204 _preview_state = {"emotion": "Neutral", "confidence": 0.0}
205 _preview_frame = None # latest RAW BGR frame (annotation now composited in camera_refresh)
206 _last_bbox = None # latest detected face bbox (x, y, w, h) in raw-frame coords; None if no face
207 DETECT_INTERVAL_SECS = 0.15
208
209 # pre-trained model paths; checks models/ first (portable), then workshop dir (dev fallback)
210 SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
211 MODELS_DIR = os.path.join(SCRIPT_DIR, "models")
212 WORKSHOP_DIR = os.path.join(SCRIPT_DIR, "..", "..", "..", "learning", "workshops") # go backwards
    ↪ to root of repo then go to learning/workshops
213
214 def find_model(local_name, workshop_subpath):
215     "Resolve a model local models/ dir first, then workshop fallback."
216     local = os.path.join(MODELS_DIR, local_name)
217     if os.path.exists(local):
218         return local
219     return os.path.join(WORKSHOP_DIR, workshop_subpath)
220
221 MODEL_JSON = find_model("model.json", os.path.join("[X]-facial-expression-detection",
    ↪ "model.json"))
222 MODEL_WEIGHTS = find_model("model_weights.weights.h5",
    ↪ os.path.join("[X]-facial-expression-detection", "model_weights.weights.h5"))
223 HAAR_CASCADE = find_model("haarcascade_frontalface_default.xml", os.path.join("[X]-ws-10",
    ↪ "haarcascade_frontalface_default.xml"))
224 SPEECH_MODEL = os.path.join(SCRIPT_DIR, "speech_emotion_model.pkl")
225 VOSK_MODEL_DIR = os.path.join(MODELS_DIR, "vosk-model-small-en-us-0.15")
226
227 # Vosk wake-word; "Pepper" or "Gaze"
228 _vosk_model = None
229 if _vosk_import_ok:
230     try:
231         _vosk_model = VoskModel(VOSK_MODEL_DIR)
232         print("[booting] vosk wake-word model loaded", flush=True)
233     except Exception as _vosk_err:
234         print(f"[booting] vosk failed to load ({_vosk_err}); wake-word gate disabled", flush=True)
235
236 RESPONSE_TIME_BASELINE = 15.0 # seconds; sits below 20s recording cap
237 CORRECTNESS_WINDOW = 5 # rolling window size
238 CORRECTNESS_FLOOR = 0.4 # below: ease off
239 CORRECTNESS_CEILING = 0.8 # above: ramp up
240 SILENCE_THRESHOLD = 2 # consecutive non-responses before intervention
241
242 SAVE_FILE = os.path.join(SCRIPT_DIR, "gaze_save.json")
243
244 # ensure AI key is set
245 if not os.getenv("OPENAI_API_KEY", "").strip():
246     raise SystemExit("ERROR: OPENAI_API_KEY not set. Add it to .env")
247 client = OpenAI()
248
249

```



```

250 class FacialExpressionModel:
251     "Pre-trained CNN: 7-classed emotion classifier (48x48 greyscale input)."
```

252

```

253     EMOTIONS = ["Angry", "Disgust", "Fear", "Happy", "Neutral", "Sad", "Surprise"]
254
255     def __init__(self, model_json_path, model_weights_path):
256         with open(model_json_path, "r") as f:
257             self.model = model_from_json(f.read())
258             self.model.load_weights(model_weights_path)
259             self.model.make_predict_function()
260
261     def predict(self, img):
262         "Return (emotion_label, confidence) from the (1, 48, 48, 1) array."
263         preds = self.model.predict(img, verbose=0)
264         idx = np.argmax(preds)
265         return self.EMOTIONS[idx], float(preds[0][idx])
266
267
268 class SpeechEmotionModel:
269     "Pre-trained MLP for vocal emotion classification (WS-08)."
```

270

```

271     EMOTIONS = ["calm", "happy", "fearful", "disgust"]
272
273     def __init__(self, model_path):
274         self.model = joblib.load(model_path)
275
276     @staticmethod # static because it's also used independently in classify_speech_emotion()
277     def extract_features(wav_path: str):
278         "Extract the same MFCC/chroma/mel feature vector used in WS-08 training."
279         with sf.SoundFile(wav_path) as sound_file:
280             audio = sound_file.read(dtype="float32")
281             sample_rate = sound_file.samplerate
282
283             if audio.ndim > 1:
284                 audio = audio.mean(axis=1)
285
286             # peak-normalise; helps quieter speakers
287             audio = librosa.util.normalize(audio)
288
289             n_fft = 2048
290             if len(audio) < n_fft:
291                 return None
292
293             stft = np.abs(librosa.stft(audio, n_fft=n_fft))
294
295             mfccs = np.mean(librosa.feature.mfcc(
296                 y=audio, sr=sample_rate, n_mfcc=40).T, axis=0).flatten()
297
298             chroma = np.mean(librosa.feature.chroma_stft(
299                 S=stft, sr=sample_rate).T, axis=0).flatten()
300
301             mel = np.mean(librosa.feature.melspectrogram(
302                 y=audio, sr=sample_rate).T, axis=0).flatten()
303
304             return np.concatenate([mfccs, chroma, mel])
305
306     def predict(self, wav_path: str) -> tuple[str, float]:
307         "Return (emotion_label, confidence) from a WAV file."
308         features = self.extract_features(wav_path)
```

```

309         if features is None:
310             return "neutral", 0.0
311
312         features = features.reshape(1, -1)
313         label = self.model.predict(features)[0]
314         proba = self.model.predict_proba(features)[0]
315         confidence = float(np.max(proba))
316         return label, confidence
317
318 def classify_speech_emotion(speech_model, wav_path: str) -> tuple[str, float]:
319     "Classify the vocal emotion from a WAV file."
320     if speech_model is None:
321         return "neutral", 0.0
322     if INPUT_IS_LOCAL and not has_real_speech(wav_path):
323         return "neutral", 0.0
324     try:
325         return speech_model.predict(wav_path)
326     except Exception as e:
327         print(f" Speech emo classify failed: {e}; defaulting to neutral")
328         return "neutral", 0.0
329
330
331 class Difficulty(Enum):
332     EASY = 1
333     MEDIUM = 2
334     HARD = 3
335
336 class InferredState(Enum):
337     THRIVING = "thriving"
338     COMFORTABLE = "comfortable"
339     STRUGGLING = "struggling"
340     DISENGAGED = "disengaged" # thus re-engage
341     FRUSTRATED = "frustrated"
342
343 class GameType(Enum):
344     NUMBERS = "numbers"
345     LETTERS = "letters"
346
347 BASE_SYSTEM_PROMPT = """
348 You are GAZE: an stroke-assitive *social-companionistic* robot running on a NAO humanoid robot.
349 You are a therapeutic companion first, and an engaging game host second.
350
351 CONVERSATION GUIDELINES:
352 - Refer to yourself ONLY in the first person ('I', 'me', 'your companion').
353 - Have natural, flowing conversations with the user.
354 - You can play countdown-style games (numbers rounds and letters rounds) when the moment feels
355   ↪ right or the user asks, but do NOT force a game every single turn.
356 - Each user message includes real-time emotional signals (facial expression, vocal emotion,
357   ↪ volume, response time). Use these signals to adapt your tone and approach naturally; doN'T
358   ↪ mention the signals explicitly.
359 - Keep responses concise: 2-3 sentences maximum. Your words are spoken aloud via text-to-speech,
360   ↪ so brevity is essential.
361 - If a game is active, acknowledge the user's answer before moving on.
362 - If the user seems disengaged, try a different topic or suggest a game. (re-engage them)
363 - If the user asks for more time to think, call request_more_time and respond understandably.
364 - If the user says the game is too hard or too easy, adjust the difficulty naturally in your next
365   ↪ generate_game_question call.
366 - Be genuinely present; you are the user's social companion.
367 """

```

```

363
364 # @dataclass: typed fields auto-become __init__ params
365
366 @dataclass
367 class GameState:
368     "Essentially tracks whether a countdown game is currently active."
369     active: bool = False
370     current_question: str = ""
371     current_answer: str = ""
372     category: str = ""
373     turn_count: int = 0
374     waiting: bool = False # user asked for more time to think
375     last_answer_checked: bool = False # was a game answer checked this turn?
376     last_answer_correct: bool = False # result of the last answer check
377     # snapshot of the answered round; survives a same-chain generate_game_question overwrite
378     answered_question: str = ""
379     answered_answer: str = ""
380     answered_game_type: Optional[GameType] = None
381     answered_difficulty: Optional[Difficulty] = None
382
383 @dataclass
384 class RoundResult:
385     "Record of a single game round."
386     round_number: int
387     game_type: GameType
388     difficulty: Difficulty
389     question: str
390     user_answer: str
391     correct: bool
392     response_time: float
393     facial_expression: str
394     expression_confidence: float
395     vocal_emotion: str
396     vocal_emotion_confidence: float
397     volume_rms: float # speech loudness (arousal signal)
398     inferred_state: InferredState
399     timestamp: float = field(default_factory=time.time)
400
401 @dataclass
402 class AdaptiveDecision:
403     "Output of the adaptive engine (what to do next)"
404     difficulty: Difficulty
405     game_type: GameType
406     inferred_state: InferredState
407     switch_game: bool
408     give_hint: bool
409     give_encouragement: bool
410     tone: str # "encouraging" / "celebratory" / "calm" / "energetic" / "neutral"
411
412
413 class AdaptiveEngine:
414     "Takes all five input signals and *infers* the user's real state. The adaptive engine also
415     ↪ evaluates if its previous adaptation worked, feeding that evaluation into the next
416     ↪ round's prompt."
417
418     def __init__(self):
419         self.history: list[RoundResult] = []
420         self.current_difficulty = Difficulty.MEDIUM
421         self.current_game = GameType.NUMBERS

```

```

420     self.consecutive_silences = 0
421     self.consecutive_correct = 0
422     self.consecutive_wrong = 0
423     self.games_played: dict[GameType, int] = {g: 0 for g in GameType}
424     self.game_switch_count = 0
425     self.adaptation_log: list[dict] = []
426     self.total_correct = 0
427     self.total_rounds_played = 0 # cumulative across resumed sessions
428     self.best_streak = 0
429     # recent-questions blocklist; cap 10
430     self.recent_questions: list[str] = []
431     self.recent_answers: list[str] = [] # answer-level dedup; catches mode-collapsed targets
432     ↪ even when GPT rephrases the question text
433     self.recent_game_types: list[str] = [] # game-type rotation hint; avoids 5-in-a-row Numbers
434     ↪ games
435     # adaptive think-budget; per-round baselines
436     self.think_budget_secs = float(RECORD_MAX_SECS) # hard ceiling
437     self.silence_tolerance_secs = float(SILENCE_DURATION) # post-speech silence
438     self.no_speech_max_secs = 5.0 # give up if no speech at all
439
440 @property
441 def round_number(self) -> int:
442     return len(self.history) + 1
443
444 def rolling_correctness(self) -> float:
445     recent = self.history[-CORRECTNESS_WINDOW:]
446     if not recent:
447         return 0.5 # no data -> assume middle (neutral)
448     return sum(1 for r in recent if r.correct) / len(recent)
449
450 VOLUME_QUIET = 200 # below this -> low arousal (quiet/disengaged)
451 VOLUME_LOUD = 2000 # above this -> high arousal (excited/frustrated)
452
453 def infer_state(self, expression: str, response_time: float,
454                 correct: bool, answer_text: str,
455                 vocal_emotion: str = "neutral",
456                 vocal_conf: float = 0.0,
457                 volume_rms: float = 0.0) -> InferredState:
458     """
459     Infer the user's state from all signals.
460     Face is primary; voice is only derived when face is Neutral and
461     the voice signal is high-confidence and not "fearful" (the MLP
462     collapses to "fearful" in silence).
463     """
464     correctness = self.rolling_correctness()
465     clean = answer_text.strip().lower()
466     is_silent = (not clean or clean in {"i don't know", "skip", "pass", "next"}) # if no
467     ↪ meaningful input || input matches a skip-command phrase in the set (set over list
468     ↪ because it's faster) then indeed silent (True)
469
470     # Arousal bounds calibrated against ambient noise
471     high_arousal = volume_rms > self.VOLUME_LOUD
472     low_arousal = 0 < volume_rms < self.VOLUME_QUIET
473
474     # voice trusted only when not-fearful
475     trust_voice = (vocal_conf >= 0.9 and vocal_emotion != "fearful")

```

```

475     if is_silent:
476         self.consecutive_silences += 1
477     else:
478         self.consecutive_silences = 0
479     if correct:
480         self.consecutive_correct += 1
481         self.consecutive_wrong = 0
482     else:
483         self.consecutive_wrong += 1
484         self.consecutive_correct = 0
485
486     # 1- FACE-PRIMARY RULES: these fire before voice is ever consulted
487
488     # thriving: good performance + fast responses
489     if (correctness >= CORRECTNESS_CEILING and response_time < RESPONSE_TIME_BASELINE * 0.5):
490         return InferredState.THRIVING
491     if expression == "Angry" and correct and response_time < RESPONSE_TIME_BASELINE * 0.6:
492         return InferredState.COMFORTABLE
493
494     # disengaged: silence + slow + poor performance
495     if self.consecutive_silences >= SILENCE_THRESHOLD:
496         return InferredState.DISENGAGED
497     if (expression == "Neutral"
498         and response_time > RESPONSE_TIME_BASELINE
499         and correctness < 0.5):
500         return InferredState.DISENGAGED
501     if (low_arousal and expression == "Neutral"
502         and response_time > RESPONSE_TIME_BASELINE * 0.8):
503         return InferredState.DISENGAGED
504
505     # frustrated: negative face + poor performance
506     if expression in ("Angry", "Disgust") and correctness < CORRECTNESS_FLOOR:
507         return InferredState.FRUSTRATED
508     if self.consecutive_wrong >= 3 and expression in ("Angry", "Sad", "Fear"):
509         return InferredState.FRUSTRATED
510     if (high_arousal and expression in ("Angry", "Disgust", "Fear")
511         and correctness < CORRECTNESS_FLOOR):
512         return InferredState.FRUSTRATED
513
514     # struggling: sadness + slow; poor correctness; fear + wrong
515     if expression == "Sad" and response_time > RESPONSE_TIME_BASELINE * 0.7:
516         return InferredState.STRUGGLING
517     if correctness < CORRECTNESS_FLOOR:
518         return InferredState.STRUGGLING
519     if expression == "Fear" and not correct:
520         return InferredState.STRUGGLING
521
522     # 2- voice tie-breakers; only when face neutral
523     if expression == "Neutral" and trust_voice:
524         if vocal_emotion == "happy" and correct and correctness >= CORRECTNESS_CEILING:
525             return InferredState.THRIVING
526         if vocal_emotion == "calm" and correctness >= 0.5:
527             return InferredState.COMFORTABLE
528
529     return InferredState.COMFORTABLE # default: face gave no negative signal, performance is
530         ⇒ holding
531
532     # core-decision function
533     def decide(self, expression: str, expression_conf: float,

```

```

533         response_time: float, correct: bool,
534         answer_text: str,
535         vocal_emotion: str = "neutral",
536         vocal_conf: float = 0.0,
537         volume_rms: float = 0.0) -> AdaptiveDecision:
538     "Return what to do next based on the inferred state."
539     state = self.infer_state(expression, response_time, correct, answer_text,
540                             vocal_emotion, vocal_conf=vocal_conf,
541                             volume_rms=volume_rms)
542     correctness = self.rolling_correctness()
543
544     new_difficulty = self.current_difficulty
545     new_game = self.current_game
546     switch_game = False
547     give_hint = False
548     give_encouragement = False
549     tone = "neutral"
550
551     if state == InferredState.THRIVING:
552         if self.current_difficulty != Difficulty.HARD:
553             new_difficulty = Difficulty(self.current_difficulty.value + 1)
554             tone = "energetic"
555         if self.consecutive_correct >= 3:
556             give_encouragement = True # acknowledge streak
557
558     elif state == InferredState.COMFORTABLE:
559         if correctness > 0.7 and self.current_difficulty != Difficulty.HARD:
560             new_difficulty = Difficulty(self.current_difficulty.value + 1)
561             tone = "neutral"
562
563     elif state == InferredState.STRUGGLING:
564         if self.current_difficulty != Difficulty.EASY:
565             new_difficulty = Difficulty(self.current_difficulty.value - 1)
566             give_hint = True
567             give_encouragement = True
568             tone = "encouraging"
569
570     elif state == InferredState.FRUSTRATED:
571         new_difficulty = Difficulty.EASY
572         give_encouragement = True
573         tone = "calm"
574         if self.consecutive_wrong >= 3:
575             switch_game = True
576             new_game = self.pick_different_game()
577
578     elif state == InferredState.DISENGAGED:
579         tone = "energetic" # robot be engaging
580         give_encouragement = True
581         if self.consecutive_silences >= 3:
582             switch_game = True
583             new_game = self.pick_different_game()
584
585     self.current_difficulty = new_difficulty
586     if switch_game:
587         self.current_game = new_game
588         self.game_switch_count += 1
589
590     self.adaptation_log.append({
591         "round": self.round_number,

```

```

592         "state": state.value,
593         "action": {"difficulty": new_difficulty.name,
594                   "switch": switch_game,
595                   "hint": give_hint,
596                   "encouragement": give_encouragement},
597     })
598
599     return AdaptiveDecision(
600         difficulty=new_difficulty, game_type=self.current_game,
601         inferred_state=state, switch_game=switch_game,
602         give_hint=give_hint, give_encouragement=give_encouragement,
603         tone=tone,
604     )
605
606     def recommend_think_budget(self, state: InferredState, expression: str,
607                               prev_response_time: float,
608                               consecutive_silences: int, waiting: bool
609                               ) -> tuple[float, float, float]:
610         """
611         Choose how long to wait for the user this turn.
612         Slower or struggling users get more time, so a long pause does not
613         flip the system to "disengaged" too quickly.
614         Returns (no_speech_max, silence_secs, record_max_secs).
615         """
616         # baseline budget (fast-track: thriving / comfortable)
617         no_speech_max = 5.0
618         silence_secs = float(SILENCE_DURATION)
619         record_max_secs = float(RECORD_MAX_SECS)
620
621         # round 1: stroke-recovery silence tolerance
622         if not self.history:
623             no_speech_max = max(no_speech_max, 7.0)
624             silence_secs = max(silence_secs, 2.5)
625             record_max_secs = max(record_max_secs, 15.0)
626
627         if state in (InferredState.STRUGGLING, InferredState.FRUSTRATED, InferredState.DISENGAGED):
628             no_speech_max = 8.0
629             silence_secs = 2.5
630             record_max_secs = 18.0
631
632         # graduated extension before disengaged flip
633         if consecutive_silences > 0:
634             no_speech_max = max(no_speech_max, 5.0 + consecutive_silences * 1.5)
635             silence_secs = max(silence_secs, 1.5 + consecutive_silences * 0.5)
636
637         if expression in ("Sad", "Fear"):
638             no_speech_max = max(no_speech_max, 7.0)
639             silence_secs = max(silence_secs, 2.0)
640
641         if prev_response_time > RESPONSE_TIME_BASELINE:
642             no_speech_max = max(no_speech_max, 7.0)
643             silence_secs = max(silence_secs, 2.0)
644
645         # LLM-flagged waiting: *honour* the signal but additively, so other cues stay weighted
646         if waiting:
647             no_speech_max += 3.0
648             silence_secs += 1.0
649             record_max_secs += 5.0
650

```

```

651         # defensive ceiling 20s; under CMD_TIMEOUT
652         record_max_secs = min(record_max_secs, 20.0)
653
654         self.no_speech_max_secs = no_speech_max
655         self.silence_tolerance_secs = silence_secs
656         self.think_budget_secs = record_max_secs
657
658         return no_speech_max, silence_secs, record_max_secs
659
660     def record_round(self, result: RoundResult):
661         self.history.append(result)
662         self.total_rounds_played += 1
663         self.games_played[result.game_type] = (
664             self.games_played.get(result.game_type, 0) + 1
665         )
666         if result.correct:
667             self.total_correct += 1
668         self.best_streak = max(self.best_streak, self.consecutive_correct)
669
670     def pick_different_game(self) -> GameType:
671         if self.current_game == GameType.NUMBERS:
672             return GameType.LETTERS
673         return GameType.NUMBERS
674
675     def get_session_summary(self) -> dict:
676         if not self.history:
677             return {"rounds": 0}
678         total = len(self.history)
679         correct = sum(1 for r in self.history if r.correct)
680         return {
681             "rounds": total,
682             "correct": correct,
683             "accuracy": round(correct / total, 2),
684             "avg_response_time": round(sum(r.response_time for r in self.history) / total, 1),
685             "games_played": {g.value: c for g, c in self.games_played.items() if c > 0},
686             "game_switches": self.game_switch_count,
687             "best_streak": self.best_streak,
688             "final_difficulty": self.current_difficulty.name,
689         }
690
691     # self-evaluative adaptation
692
693     def evaluate_adaptation(self) -> Optional[str]:
694         "Evaluate whether the previous round's adaptation worked."
695         if len(self.adaptation_log) < 2 or len(self.history) < 2:
696             return None
697
698         prev_strategy = self.adaptation_log[-2]
699         curr_strategy = self.adaptation_log[-1]
700         prev_round = self.history[-2]
701         curr_round = self.history[-1]
702
703         prev_state = prev_strategy["state"]
704         curr_state = curr_strategy["state"]
705         prev_action = prev_strategy["action"]
706
707         evaluations = []
708
709         if prev_state in ("struggling", "frustrated"):

```



```

710         if curr_round.correct and not prev_round.correct:
711             evaluations.append("Previous adaptation WORKED: lowered difficulty and user answered
              ↳ correctly this round (was incorrect before).")
712         elif not curr_round.correct:
713             evaluations.append("Previous adaptation DID NOT HELP YET: user still struggling
              ↳ despite easier difficulty. Consider providing more support.")
714
715         # Did a difficulty increase overshoot for a thriving user?
716         if prev_state == "thriving" and prev_action["difficulty"] == "HARD":
717             if curr_round.correct:
718                 evaluations.append("Previous adaptation WORKED: increased difficulty and user is
              ↳ still performing well.")
719             elif not curr_round.correct:
720                 evaluations.append("Previous adaptation OVERSHOT: increased difficulty but user got
              ↳ it wrong. Might need to ease back.")
721
722         # Did a re-engagement attempt work for a disengaged user?
723         if prev_state == "disengaged":
724             if curr_state != "disengaged":
725                 evaluations.append(f"Previous adaptation WORKED: user was disengaged but is now
              ↳ {curr_state}. Re-engagement WAS effective.")
726             else:
727                 evaluations.append("Previous adaptation DID NOT HELP: user remains disengaged. Try a
              ↳ different approach or switch game type.")
728
729         if prev_action.get("switch"):
730             if curr_state in ("thriving", "comfortable"):
731                 evaluations.append(
732                     "Game switch WORKED: user transitioned to a positive state."
733                 )
734             elif curr_state in ("struggling", "frustrated", "disengaged"):
735                 evaluations.append(
736                     "Game switch DID NOT HELP: user is still in a negative state."
737                 )
738
739         if (prev_action.get("encouragement")
740             and prev_state in ("struggling", "frustrated")
741             and curr_round.response_time < prev_round.response_time):
742             evaluations.append(
743                 "Encouragement appears effective: user responded faster this round."
744             )
745
746         if not evaluations:
747             return None
748
749         return ("Adaptation evaluation from previous round:\n"
750                + "\n".join(f"- {e}" for e in evaluations))
751
752
753     GAME_DESCRIPTIONS = {
754         GameType.NUMBERS: """
755         a Countdown-style numbers round. Pick SIX numbers by sampling from {1 to 100}; vary the mix
              ↳ each round so no two rounds in a row feel identical.
756         Pick a TARGET in the range 50-999 (anywhere in that range, NOT always around 100), reachable
              ↳ from the chosen six via +, -, *, /.
757         The user must combine the given numbers with those operators to reach the target. Each number
              ↳ may be used at most once and it should be humanly solveable""",
758         GameType.LETTERS: """
759         a Countdown-style letters round: give the user a set of 9 random letters (a mix of vowels and

```

```

    ↪ consonants; vary the letter set every round) and ask them to form the longest word
    ↪ possible using only those letters.
760     Each letter can only be used once"",
761 }
762
763 DIFFICULTY_DESCRIPTIONS = {
764     Difficulty.EASY: "easy (straightforward, common knowledge, single-step)",
765     Difficulty.MEDIUM: "medium (requires some thought, moderately specific)",
766     Difficulty.HARD: "hard (obscure, multi-step, requires deep knowledge)",
767 }
768
769 # SSH AND PEPPER ROBOT HELPERS: NON-INDENTED PYTHON STRINGS
770 # (adapted from lab-robot-code-fin.py i.e. when
771 # we tested OpenAI on the NAO robot perhaps too early)
772
773 def ssh_connect():
774     "Open SSH connection to Pepper and return the client."
775     ssh = paramiko.SSHClient()
776     ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
777     ssh.connect(NAO_IP, username=NAO_USER, password=NAO_PASS, timeout=SSH_TIMEOUT)
778     return ssh
779
780 def nao_run(ssh, code):
781     "Execute a Python 2 snippet on Pepper via SSH."
782     escaped = code.replace("'", "'\\'")
783     try:
784         _, stdout, _ = ssh.exec_command(f"python -c '{escaped}'", timeout=CMD_TIMEOUT)
785         return stdout.read().decode().strip()
786     except Exception as e:
787         print(f" Pepper SSH exec_command dropped: {e}")
788         return ""
789
790 #listens silent
791 #not triggered by noise
792 def nao_calibrate_ambient(ssh) -> int:
793     "Calibrate the mic energy threshold to the room."
794     try:
795         # col 0 only; Pepper rejects indented python -c
796         raw = nao_run(ssh, f"")
797 from naoqi import ALProxy
798 import time
799
800 audio = ALProxy("ALAudioDevice", "127.0.0.1", 9559)
801 samples = []
802 start = time.time()
803 while (time.time() - start) < {CALIBRATION_SECS}:
804     # all four mics; side speakers else missed
805     try:
806         front = audio.getFrontMicEnergy()
807         left = audio.getLeftMicEnergy()
808         right = audio.getRightMicEnergy()
809         rear = audio.getRearMicEnergy()
810         samples.append(max(front, left, right, rear))
811     except Exception:
812         # older firmwares may expose only getFrontMicEnergy; fall back gracefully
813         samples.append(audio.getFrontMicEnergy())
814     time.sleep(0.2)
815
816 if samples:

```

```

817     avg = sum(samples) / len(samples)
818     print(int(avg))
819 else:
820     print(0)
821     """
822     ambient = int(raw) if raw.strip().isdigit() else 0
823     threshold = ambient + ENERGY_BUFFER
824     print(f" Ambient energy: {ambient}, speech threshold: {threshold}")
825     return threshold
826 except Exception as e:
827     print(f" Calibration failed ({e}), using default threshold: {DEFAULT_ENERGY_THRESHOLD}")
828     return DEFAULT_ENERGY_THRESHOLD
829
830 def nao_record(ssh, energy_threshold: int = DEFAULT_ENERGY_THRESHOLD,
831               record_max_secs: float = RECORD_MAX_SECS,
832               silence_secs: float = SILENCE_DURATION):
833     """Record audio on Pepper with dynamic silence detection.
834     - get robot's front microphone (getFrontMicEnergy) to stop recording early if silence
835       ↳ detected
836     - calibrated energy threshold to avoid false positives from ambient noise
837     - if getFrontMicEnergy is unsupported (e.g. older firmware), fall back to a safe
838       ↳ fixed-duration recording to ensure the demo still works, albeit without silence
839       ↳ detection
840     """
841     # fixed-duration fallback for old firmware
842     nao_run(ssh, f"""
843 from naoqi import ALProxy
844 import time
845
846 rec = ALProxy("ALAudioRecorder", "127.0.0.1", 9559)
847
848 rec.stopMicrophonesRecording()
849 rec.startMicrophonesRecording("{REMOTE_WAV}", "wav", 16000, [1, 1, 1, 1])
850
851 try:
852     audio = ALProxy("ALAudioDevice", "127.0.0.1", 9559)
853
854     speech_detected = False
855     silence_start = None
856     start = time.time()
857     threshold = {energy_threshold}
858
859     while True:
860         elapsed = time.time() - start
861
862         # hard ceiling; never exceed max duration
863         if elapsed >= {record_max_secs}:
864             break
865
866         # all four mics; side speakers else missed
867         try:
868             energy = max(
869                 audio.getFrontMicEnergy(),
870                 audio.getLeftMicEnergy(),
871                 audio.getRightMicEnergy(),
872                 audio.getRearMicEnergy(),
873             )
874         except Exception:
875             energy = audio.getFrontMicEnergy() # firmware fallback

```

```

873
874     if elapsed < {RECORD_MIN_SECS}:
875         # minimum recording period
876         if energy > threshold:
877             speech_detected = True
878             time.sleep({SILENCE_POLL_SECS})
879             continue
880
881     if energy > threshold:
882         speech_detected = True
883         silence_start = None
884     else:
885         if speech_detected and silence_start is None:
886             silence_start = time.time()
887         if speech_detected and silence_start is not None:
888             if (time.time() - silence_start) >= {silence_secs}:
889                 break
890
891     time.sleep({SILENCE_POLL_SECS})
892
893 except Exception as e:
894     # firmware fallback; if getFrontMicEnergy() unsupported, fixed-duration recording keeps the
895     ↪ demo alive
896     print(" [Silence detection failed: " + str(e) + "] Falling back to fixed-duration recording")
897     time.sleep({record_max_secs})
898
899 rec.stopMicrophonesRecording()
900
901 sftp = ssh.open_sftp()
902 sftp.get(REMOTE_WAV, LOCAL_WAV)
903 sftp.close()
904 # no gain stage; amplified room noise
905
906 # WAV layout diagnostic; four-mic recording may land multi-channel
907 try:
908     with wave.open(LOCAL_WAV, "rb") as wf:
909         secs = wf.getnframes() / float(wf.getframerate())
910         print(f" WAV debug: channels={wf.getnchannels()}, rate={wf.getframerate()},
911             ↪ frames={wf.getnframes()}, secs={secs:.2f}")
912 except Exception as e:
913     print(f" WAV debug failed: {e}")
914
915 # collapse to mono 16 kHz so downstream gates see a canonical layout
916 force_mono_16k_wav(LOCAL_WAV)
917
918 def force_mono_16k_wav(path: str) -> None:
919     "Convert `path` to mono 16 kHz int16 PCM."
920     try:
921         audio, sr = sf.read(path, dtype="float32")
922         if audio.size == 0:
923             return
924         if audio.ndim > 1:
925             # loudest mic; mics not phase-aligned
926             rms_by_channel = np.sqrt(np.mean(audio.astype(np.float64) ** 2, axis=0))
927             audio = audio[:, int(np.argmax(rms_by_channel))]
928         if sr != LOCAL_SAMPLE_RATE:
929             audio = librosa.resample(audio, orig_sr=sr, target_sr=LOCAL_SAMPLE_RATE)
930         audio = np.clip(audio, -1.0, 1.0)
931         sf.write(path, audio, LOCAL_SAMPLE_RATE, subtype="PCM_16")

```

```

930     except Exception as e:
931         print(f" Mono conversion skipped ({e})")
932
933     #responses arrive in one block no splitting
934     #delivered with no pauses
935     def split_into_sentences(text: str) -> list[str]:
936         "Split dialogue into sentences for speech delivery."
937         raw_segments = re.split(r'(?<=[.!?])\s+', text.strip())
938         sentences = []
939         for seg in raw_segments:
940             for line in seg.split("\n"):
941                 cleaned = line.strip()
942                 if cleaned:
943                     sentences.append(cleaned)
944         return sentences if sentences else [text.strip()]
945
946     _TTS_REPLACEMENTS = { # prevent unicode characters from breaking Pepper Text-To-Speech
947         "'": "'", "' ': '",
948         "\"": "\"", "\" ': '",
949         "-": "-", "- ": "-", "- ": "-",
950         "...": "...", " ": " ",
951     }
952
953     def clean_for_tts(text: str) -> str:
954         "Strip gesture tags and Unicode escapes for Pepper TTS."
955         text = text or ""
956         text = re.sub(r'\[gesture:\w+\]', '', text)
957         text = unicodedata.normalize("NFKC", text)
958         for bad, good in _TTS_REPLACEMENTS.items():
959             text = text.replace(bad, good)
960         # animated-speech tags
961         text = re.sub(r'\^[A-Za-z_]+(?:\[([~])*\])?$', '', text)
962         # ALTextToSpeech tags
963         text = re.sub(r'\\[A-Za-z]{2,8}=[^\\]*\\', '', text)
964         # literal \uXXXX leftovers
965         text = re.sub(r'\\u[0-9a-fA-F]{4}', '', text)
966         # control characters
967         text = re.sub(r'[\x00-\x1f\x7f-\x9f]', ' ', text)
968         return re.sub(r'\s+', ' ', text).strip()
969
970     #single ssh calls
971     def nao_say(ssh, text):
972         "Speak text on Pepper with sentence-level pausing."
973         text = clean_for_tts(text)
974         sentences = split_into_sentences(text)
975         safe_sentences = json.dumps(sentences)
976
977         nao_run(ssh, f"""
978 from naoqi import ALProxy
979 import time
980
981 try:
982     tts = ALProxy("ALTextToSpeech", "127.0.0.1", 9559)
983     sentences = {safe_sentences}
984
985     for i, sentence in enumerate(sentences):
986         tts.say(sentence)
987         if i < len(sentences) - 1:
988             time.sleep(0.4)

```

```

989 except Exception as e:
990     print(" [TTS failed] " + str(e))
991     """
992
993 #animations with speech
994 def nao_say_animated(ssh, text):
995     "Try animated speech; fall back to plain TTS."
996     safe = json.dumps(text)
997     try:
998         nao_run(ssh, f"""
999 from naoqi import ALProxy
1000 ALProxy("ALAnimatedSpeech","127.0.0.1",9559).say({safe})
1001 """)
1002     except Exception as e:
1003         print(f" Animated speech failed mid-sentence: {e}")
1004         nao_say(ssh, text)
1005
1006 def nao_capture_image(ssh):
1007     "Capture a photo from Pepper's camera and download it. Kept as a fallback for one-off stills;
1008     ↪ the live dashboard now uses pepper_video_loop()."
1009     nao_run(ssh, f"""
1010 from naoqi import ALProxy
1011 pc = ALProxy("ALPhotoCapture","127.0.0.1",9559)
1012 pc.setResolution(2)
1013 pc.setPictureFormat("jpg")
1014 pc.takePicture("{os.path.dirname(REMOTE_IMG)}/",
1015     ↪ "{os.path.splitext(os.path.basename(REMOTE_IMG))[0]}")
1016 """)
1017     sftp = ssh.open_sftp()
1018     sftp.get(REMOTE_IMG, LOCAL_IMG)
1019     sftp.close()
1020
1021 PEPPER_VIDEO_PORT = 9558
1022 _pepper_video_channel = None
1023
1024 def pepper_video_loop(ssh):
1025     """New persistent ALVideoDevice on Pepper streaming length-prefixed RGB frames over
1026     TCP (Transmission Control Protocol)"""
1027     global _pepper_video_channel
1028
1029     code = textwrap.dedent('''
1030 from naoqi import ALProxy
1031 import socket, struct, time
1032
1033 HOST = "0.0.0.0"
1034 PORT = {port}
1035
1036 cam = ALProxy("ALVideoDevice", "127.0.0.1", 9559)
1037 # camera_id=0 (top), resolution=1 (320x240), colour_space=11 (RGB), fps=10
1038 name = cam.subscribeCamera("gaze_video", 0, 1, 11, 10)
1039
1040 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
1041 server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
1042 server.bind((HOST, PORT))
1043 server.listen(1)
1044
1045 conn, addr = server.accept()
1046
1047 try:

```

```

1046         while True:
1047             img = cam.getImageRemote(name)
1048             if img is None:
1049                 time.sleep(0.05)
1050                 continue
1051             width = img[0]
1052             height = img[1]
1053             data = img[6]
1054             payload = struct.pack("!II", width, height) + data
1055             conn.sendall(struct.pack("!I", len(payload)) + payload)
1056             time.sleep(0.1)
1057         finally:
1058             try:
1059                 cam.unsubscribe(name)
1060             except Exception:
1061                 pass
1062             try:
1063                 conn.close()
1064             except Exception:
1065                 pass
1066             server.close()
1067     '').format(port=PEPPER_VIDEO_PORT)
1068
1069     escaped = code.replace('"', '\\\\"')
1070     # async; nao_run() reads stdout, would otherwise block
1071     _stdin, stdout, _stderr = ssh.exec_command(f'python -c '{escaped}''')
1072     _pepper_video_channel = stdout.channel # keep referenced so Paramiko doesn't GC the channel
1073     ↳ out from under the remote process
1074
1075 def _recv_exact(sock, n):
1076     "Receive exactly n bytes; raise if the socket closes mid-frame."
1077     data = b""
1078     while len(data) < n:
1079         packet = sock.recv(n - len(data))
1080         if not packet:
1081             raise ConnectionError("Socket closed whilst receiving Pepper camera frame")
1082         data += packet
1083     return data
1084
1085 def pepper_camera_receive_loop(pepper_ip: str):
1086     "Pull length-prefixed RGB frames off Pepper:9558 into _preview_frame for detect_thread_loop to
1087     ↳ consume; ten 1-second connect-retries because Pepper-side bind() needs a beat to land
1088     ↳ after exec_command."
1089     global _preview_frame
1090     sock = None
1091     for attempt in range(10):
1092         try:
1093             sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
1094             sock.connect((pepper_ip, PEPPER_VIDEO_PORT))
1095             print(f" Pepper video socket connected on attempt {attempt + 1}.")
1096             break
1097         except (ConnectionRefusedError, OSError) as e:
1098             print(f" Pepper video connect attempt {attempt + 1} failed ({e}); retrying...")
1099             try:
1100                 sock.close()
1101             except Exception:

```

```

1102         pass
1103         sock = None
1104         time.sleep(1.0)
1105     if sock is None:
1106         print(" Pepper video stream unreachable; dashboard will stay black.")
1107         return
1108
1109     try:
1110         while True: # whilst running
1111             size_raw = _recv_exact(sock, 4)
1112             size = struct.unpack("!I", size_raw)[0]
1113             payload = _recv_exact(sock, size)
1114             width, height = struct.unpack("!II", payload[:8])
1115             rgb_bytes = payload[8:]
1116             rgb = np.frombuffer(rgb_bytes, dtype=np.uint8).reshape((height, width, 3))
1117             frame = cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR) # rest of pipeline is BGR (OpenCV
                  ↪ convention)
1118             with _preview_lock:
1119                 _preview_frame = frame
1120     except Exception as e:
1121         print(f" Pepper video receiver loop exited: {e}")
1122     finally:
1123         try:
1124             sock.close()
1125         except Exception:
1126             pass
1127
1128 def nao_track_face(ssh, enable=True):
1129     "Toggle face tracking on Pepper."
1130     try:
1131         if enable:
1132             nao_run(ssh, """
1133 from naoqi import ALProxy
1134 ALProxy("ALFaceDetection","127.0.0.1",9559).subscribe("gaze_face")
1135 t = ALProxy("ALTracker","127.0.0.1",9559)
1136 t.registerTarget("Face", 0.1)
1137 t.track("Face")
1138 """)
1139         else:
1140             nao_run(ssh, """
1141 from naoqi import ALProxy
1142 t = ALProxy("ALTracker","127.0.0.1",9559)
1143 t.stopTracker()
1144 t.unregisterAllTargets()
1145 try:
1146     ALProxy("ALFaceDetection","127.0.0.1",9559).unsubscribe("gaze_face")
1147 except Exception as e:
1148     print(" [Face unsubscribe failed] " + str(e))
1149 """)
1150     except Exception as e:
1151         print(f" Face tracking unavailable: {e}")
1152
1153 #leds
1154 def nao_set_leds(ssh, group, colour, duration=1.0):
1155     try:
1156         nao_run(ssh, f"""
1157 from naoqi import ALProxy
1158 ALProxy("ALLeds","127.0.0.1",9559).fadeRGB("{group}", {colour}, {duration})
1159 """)

```



```

1160     except Exception as e:
1161         print(f" LED set ignored: {e}")
1162
1163 # LOCAL MODE HELPERS
1164
1165 LOCAL_SAMPLE_RATE = 16000 # Whisper expects 16 kHz; we resample from native rate
1166
1167 LOCAL_SILENCE_RMS = 40 # default RMS; overridden by local_calibrate_ambient()
1168 _local_speech_detected = False # set by local_record(); used as transcription gate
1169 LOCAL_SILENCE_SECS = 1.2 # seconds of post-speech silence to stop recording; 1.5 →1.2
1170     ↪ (aphasia-safe)
1171 LOCAL_MIN_SECS = 1.0 # minimum recording before silence detection kicks in
1172 LOCAL_NO_SPEECH_MAX = 3.0 # stop if no speech detected at all after this many seconds; 5.0 →3.0
1173     ↪ (dominant lag)
1174 LOCAL_ENERGY_BUFFER = 60 # margin above ambient baseline for speech detection; 50 →25 to catch
1175     ↪ quiet speech
1176
1177 def local_calibrate_ambient() -> int:
1178     "Calibrate the local-testing (Mac) mic's ambient noise level; mirrors nao_calibrate_ambient()
1179     ↪ so LOCAL_MODE testing behaves like the robot."
1180
1181     global LOCAL_SILENCE_RMS
1182     dev_info = sd.query_devices(kind="input")
1183     dev_index = dev_info["index"]
1184     native_rate = int(dev_info["default_samplerate"])
1185     chunk_size = int(native_rate * 0.2)
1186     samples = []
1187
1188     print(f"Calibrating Mac mic (stay quiet for {CALIBRATION_SECS}s)...")
1189
1190     def cb(indata, frames, time_info, status): # get stable baseline from average RMS
1191         rms = (np.mean(indata.astype(np.float64) ** 2)) ** 0.5
1192         samples.append(rms)
1193
1194     with sd.InputStream(samplerate=native_rate, channels=1, dtype="int16",
1195                        blocksize=chunk_size, device=dev_index, callback=cb):
1196         time.sleep(CALIBRATION_SECS)
1197
1198     if samples:
1199         ambient = int(sum(samples) / len(samples))
1200     else:
1201         ambient = 0
1202
1203     threshold = ambient + LOCAL_ENERGY_BUFFER
1204     LOCAL_SILENCE_RMS = threshold
1205
1206     # 500 was too high for MacBook mic
1207     global VOLUME_THRESHOLD
1208     VOLUME_THRESHOLD = max(LOCAL_SILENCE_RMS * 4, 100)
1209
1210     # scale to room; static thresholds confuse loud/quiet rooms
1211     AdaptiveEngine.VOLUME_QUIET = max(ambient * 2, 200) # 200 = absolute quiet floor
1212     AdaptiveEngine.VOLUME_LOUD = max(ambient * 10, 2000)
1213
1214     print(f" Ambient RMS: {ambient}, silence threshold: {threshold}, transcription gate:
1215     ↪ {VOLUME_THRESHOLD}")
1216     print(f" Arousal thresholds: QUIET={AdaptiveEngine.VOLUME_QUIET},
1217     ↪ LOUD={AdaptiveEngine.VOLUME_LOUD}")
1218     return threshold
1219
1220

```

```

1213 def local_record(max_secs: float = RECORD_MAX_SECS,
1214                  no_speech_max: float = LOCAL_NO_SPEECH_MAX,
1215                  silence_secs: float = LOCAL_SILENCE_SECS):
1216     "Record audio from the Mac's built-in microphone to LOCAL_WAV with silence detection mirroring
        ↳ Pepper's dynamic recording."
1217     # select the default input device
1218     dev_info = sd.query_devices(kind="input")
1219     dev_index = dev_info["index"]
1220     native_rate = int(dev_info["default_samplerate"])
1221     sd.default.device = (dev_index, None)
1222     chunk_size = int(native_rate * SILENCE_POLL_SECS)
1223
1224     buffer = []
1225     speech_detected = False
1226     silence_start = None
1227     elapsed = 0.0
1228
1229     print(f" Recording from Mac mic (up to {max_secs}s, stops after silence)...")
1230
1231     def callback(indata, frames, time_info, status):
1232         buffer.append(indata.copy())
1233
1234     for attempt in range(2): #one retry if PortAudio error (if the device briefly is unavailable
        ↳ after Pepper usage)
1235         try:
1236             with sd.InputStream(samplerate=native_rate, channels=1, dtype="int16",
1237                               blocksize=chunk_size, callback=callback):
1238                 while elapsed < max_secs:
1239                     time.sleep(SILENCE_POLL_SECS)
1240                     elapsed += SILENCE_POLL_SECS
1241
1242                     if not buffer:
1243                         continue
1244                     data = buffer[-1]
1245
1246                     rms = (np.mean(data.astype(np.float64) ** 2)) ** 0.5 # compute RMS of the latest
                        ↳ chunk for real-time feedback
1247                     global _last_rms
1248                     _last_rms = rms # ' ' (U+2593 DARK SHADE) and ' ' (U+2591
                        ↳ LIGHT SHADE) extracted from Unicode 1.1 Block Elements
1249                     print(f"\r [{elapsed:.1f}s] RMS: {rms:.0f} {' ' if rms > LOCAL_SILENCE_RMS else
                        ↳ ' '}", end="", flush=True)
1250
1251                     if elapsed < LOCAL_MIN_SECS:
1252                         if rms > LOCAL_SILENCE_RMS:
1253                             speech_detected = True
1254                             continue
1255
1256                     if not speech_detected and elapsed >= no_speech_max:
1257                         break
1258
1259                     if rms > LOCAL_SILENCE_RMS:
1260                         speech_detected = True
1261                         silence_start = None
1262                     else:
1263                         if speech_detected and silence_start is None:
1264                             silence_start = elapsed
1265                         if speech_detected and silence_start is not None:
1266                             if (elapsed - silence_start) >= silence_secs:

```

```

1267         break
1268     break # stream ran to completion
1269 except sd.PortAudioError as e:
1270     print(f"\n [PortAudio error, attempt {attempt + 1}] {e}")
1271     if attempt == 0:
1272         time.sleep(0.5) # let CoreAudio release the device
1273         continue
1274     print(" Mic unavailable; saving silent recording and continuing.")
1275
1276 print()
1277
1278 if buffer:
1279     audio_native = np.concatenate(buffer).flatten().astype(np.float32) / 32768.0
1280     # resample to 16 kHz for Whisper
1281     audio_16k = librosa.resample(audio_native, orig_sr=native_rate, target_sr=LOCAL_SAMPLE_RATE)
1282     audio_int16 = (audio_16k * 32768.0).astype(np.int16)
1283 else:
1284     audio_int16 = np.zeros((0,), dtype=np.int16)
1285
1286 # speech-detected flag so the transcription gate can use it
1287 global _local_speech_detected
1288 _local_speech_detected = speech_detected
1289
1290 with wave.open(LOCAL_WAV, "wb") as wf:
1291     wf.setnchannels(1)
1292     wf.setsampwidth(2)
1293     wf.setframerate(LOCAL_SAMPLE_RATE)
1294     wf.writeframes(audio_int16.tobytes())
1295 print(f" Recording saved ({elapsed:.1f}s, speech={'yes' if speech_detected else 'no'})")
1296
1297 def local_say(text: str):
1298     "Speak text using host OS built-in TTS (macOS `say` or Windows SAPI)."
1299     text = clean_for_tts(text)
1300     try:
1301         if sys.platform == "win32":
1302             # Windows: SAPI via PowerShell; env-var avoids escaping
1303             subprocess.run(
1304                 ["powershell", "-NoProfile", "-Command",
1305                  "Add-Type -AssemblyName System.Speech;(New-Object"
1306                  "    ↳ System.Speech.Synthesis.SpeechSynthesizer).Speak($env:GAZE_TTS_TEXT)"],
1307                 env={**os.environ, "GAZE_TTS_TEXT": text},
1308                 check=True, timeout=30,
1309             )
1310         else:
1311             subprocess.run(["say", text], check=True, timeout=30)
1312     except Exception as e:
1313         print(f" Local TTS broke: {e}")
1314
1315 def say(ssh_tts, text):
1316     "Dispatch TTS to local or Pepper depending on mode."
1317     text = clean_for_tts(text)
1318     if LOCAL_MODE:
1319         local_say(text)
1320     else:
1321         nao_say(ssh_tts, text)
1322     time.sleep(0.5) # ensure text-to-speech (TTS) drains so it does not hear itself
1323
1324 def record(ssh, energy_threshold,
1325            no_speech_max: float = LOCAL_NO_SPEECH_MAX,

```

```

1325         silence_secs: float = LOCAL_SILENCE_SECS,
1326         record_max_secs: float = RECORD_MAX_SECS):
1327     """Dispatch recording to local or Pepper depending on mode;
1328     per-turn think-budget set by: AdaptiveEngine.recommend_think_budget().
1329     INPUT_IS_LOCAL routes the mic to the Mac even when LOCAL_MODE is False
1330     (gaze21 hybrid: Pepper-out, Mac-in)."""
1331     if INPUT_IS_LOCAL:
1332         local_record(max_secs=record_max_secs,
1333                     no_speech_max=no_speech_max,
1334                     silence_secs=silence_secs)
1335     else:
1336         nao_record(ssh, energy_threshold,
1337                  record_max_secs=record_max_secs,
1338                  silence_secs=silence_secs)
1339
1340     # gesture mapping; motions per game/emotional context
1341     # tags: celebrate, encourage, think, wave, calm, energetic, neutral
1342
1343     GESTURE_CODE = {
1344         "wave": """
1345 from naoqi import ALProxy
1346 import time
1347 m = ALProxy("ALMotion","127.0.0.1",9559)
1348 m.setStiffnesses("RArm", 1.0)
1349 safe = ["RShoulderPitch","RShoulderRoll","RElbowYaw","RElbowRoll","RWristYaw","RHand"]
1350 m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
1351 time.sleep(0.2)
1352 m.angleInterpolation(["RShoulderRoll"], [-0.45], [1.5], True)
1353 time.sleep(0.2)
1354 m.angleInterpolation(safe, [-0.3, -0.3, 1.0, 1.0, 0.0, 1.0], [2.0]*6, True)
1355 for _ in range(3):
1356     m.angleInterpolation(["RWristYaw"], [0.5], [1.2], True)
1357     m.angleInterpolation(["RWristYaw"], [-0.5], [1.2], True)
1358 time.sleep(0.4)
1359 m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
1360 m.setStiffnesses("RArm", 0.0)
1361 """,
1362
1363         "correct_wave": """
1364 from naoqi import ALProxy
1365 import time
1366 m = ALProxy("ALMotion","127.0.0.1",9559)
1367 m.setStiffnesses("RArm", 1.0)
1368 safe = ["RShoulderPitch","RShoulderRoll","RElbowYaw","RElbowRoll","RWristYaw","RHand"]
1369 m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
1370 time.sleep(0.2)
1371 m.angleInterpolation(["RShoulderRoll"], [-0.45], [1.5], True)
1372 time.sleep(0.2)
1373 m.angleInterpolation(safe, [-0.3, -0.3, 1.0, 1.0, 0.0, 1.0], [2.0]*6, True)
1374 for _ in range(3):
1375     m.angleInterpolation(["RWristYaw"], [0.5], [1.2], True)
1376     m.angleInterpolation(["RWristYaw"], [-0.5], [1.2], True)
1377 time.sleep(0.4)
1378 m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
1379 m.setStiffnesses("RArm", 0.0)
1380 """,
1381     }
1382
1383

```

```

1384 # LED COLOUR MAP
1385 LED_COLOURS = {
1386     InferredState.THRIVING: 0x0000FF00, # green
1387     InferredState.COMFORTABLE: 0x0000FFFF, # cyan
1388     InferredState.STRUGGLING: 0x00FFFF00, # yellow
1389     InferredState.FRUSTATED: 0x00FF0000, # red
1390     InferredState.DISENGAGED: 0x00FF00FF, # magenta
1391 }
1392
1393
1394 _STATE_COLOURS = {
1395     InferredState.THRIVING: "green",
1396     InferredState.COMFORTABLE: "blue",
1397     InferredState.STRUGGLING: "orange",
1398     InferredState.FRUSTATED: "red",
1399     InferredState.DISENGAGED: "grey",
1400 }
1401
1402 def nao_gesture(ssh, gesture_type: str):
1403     "Execute a gesture on Pepper aligned to the game context."
1404     code = GESTURE_CODE.get(gesture_type, GESTURE_CODE["wave"])
1405     try:
1406         nao_run(ssh, code)
1407     except Exception as e:
1408         print(f" Gesture {gesture_type!r} did not play: {e}")
1409
1410
1411 def measure_volume() -> float:
1412     "RMS amplitude of the WAV; soundfile-based so multi-channel files collapse to mono cleanly."
1413     try:
1414         audio, _sr = sf.read(LOCAL_WAV, dtype="float32")
1415         if audio.size == 0:
1416             return 0.0
1417         if audio.ndim > 1:
1418             audio = audio.mean(axis=1)
1419         return float(np.sqrt(np.mean((audio * 32768.0) ** 2)))
1420     except Exception as e:
1421         print(f" Volume RMS calc failed: {e}")
1422     return 0.0
1423
1424 # Whisper hallucination blacklist; pre-normalised
1425 # exact-match: short fillers that would false-positive if matched as substrings.
1426 WHISPER_HALLUCINATION_EXACT = {
1427     "you", "mm", "hmm", "uh", "um",
1428     "thank you", "thanks",
1429     " ",
1430     " ",
1431     " ",
1432     " ",
1433     " ",
1434 }
1435
1436 # fragments: if any appears anywhere in the normalised text, treat as hallucination.
1437 WHISPER_HALLUCINATION_FRAGMENTS = (
1438     "questions or comments",
1439     "questions or other problems",
1440     "post them in the comments",
1441     "in the comments section",
1442     "in the comment section",

```

```

1443     "if you like this video",
1444     "if you liked this video",
1445     "if you enjoyed the video",
1446     "if you enjoyed this video",
1447     "please subscribe",
1448     "subscribe and like",
1449     "like and subscribe",
1450     "like it and subscribe",
1451     "subscribe to our channel",
1452     "subscribe to my channel",
1453     "dont forget to subscribe",
1454     "do not forget to subscribe",
1455     "hit the like button",
1456     "smash that like button",
1457     "click the link",
1458     "link in the description",
1459     "link in the bio",
1460     "in the description below",
1461     "thanks for watching",
1462     "thank you for watching",
1463     "thank you so much for watching",
1464     "thanks so much for watching",
1465     "see you next time",
1466     "ill see you next time",
1467     "see you in the next video",
1468     "see you in the next one",
1469     # Whisper-prompt regurgitation; default prompt mentions "companion robot"
1470     "companion robot",
1471     "chats with a companion",
1472     "answers a quiz",
1473     "with a companion robot",
1474 )
1475
1476 # back-compat alias
1477 WHISPER_HALLUCINATIONS = WHISPER_HALLUCINATION_EXACT
1478
1479 def normalise_for_blacklist(text: str) -> str:
1480     "Normalise so blacklist matches independent of Whisper output."
1481     t = unicodedata.normalize("NFKC", text).lower()
1482     kept = [ch for ch in t if ch.isalnum() or ch.isspace()]
1483     return " ".join("".join(kept).split())
1484
1485 # regex blacklist; URL outros, promo boilerplate
1486 _URL_HALLUCINATION =
1487     ↪ re.compile(r"\bhttps?://|www\.\b|w+\.(?:com|org|net|au|co\.uk|google|sites)\b", re.I)
1487 _DISCLAIMER_HALLUCINATION = re.compile(r"\b(please see|visit|subscribe|like and
1488     ↪ subscribe|disclaimer|description)\b.{0,40}\b(complete|full|link|below|above)\b", re.I)
1488
1489 def is_known_hallucination(text: str) -> bool:
1490     norm = normalise_for_blacklist(text)
1491     if norm == "" or norm in WHISPER_HALLUCINATION_EXACT:
1492         return True
1493     # any fragment present anywhere in the normalised text counts
1494     if any(frag in norm for frag in WHISPER_HALLUCINATION_FRAGMENTS):
1495         return True
1496     # structural catch: Whisper leaks YouTube-description URLs on silence-amplified input
1497     if _URL_HALLUCINATION.search(text):
1498         return True
1499     if _DISCLAIMER_HALLUCINATION.search(text):

```

```

1500         return True
1501     return False
1502
1503 def has_real_speech(wav_path: str, min_speech_ms: int = 250,
1504                    threshold: float = 0.5) -> bool:
1505     "Silero VAD pre-gate; relaxed defaults so short answers (yes/Tom/six) survive."
1506     if _silero_model is None or get_speech_timestamps is None:
1507         return True
1508     try:
1509         wav = read_audio(wav_path, sampling_rate=LOCAL_SAMPLE_RATE)
1510         segments = get_speech_timestamps( # pass in the audio and model to return a list of dicts
1511                                         ↪ containing the start and end frame indices of genuine speech
1512                                         wav, _silero_model,
1513                                         sampling_rate=LOCAL_SAMPLE_RATE,
1514                                         min_speech_duration_ms=min_speech_ms,
1515                                         threshold=threshold,
1516                                         return_seconds=False,
1517                                         )
1518         return bool(segments)
1519     except Exception as e:
1520         print(f" Silero VAD check failed ({e}); falling through to Whisper")
1521         return True
1522
1523 def has_wake_word(wav_path: str) -> bool:
1524     """
1525     Vosk wake-word gate. True if "Pepper" or "Gaze" is heard in the WAV.
1526     Returns True if the Vosk model didn't load (optional dep).
1527     """
1528     if _vosk_model is None or KaldiRecognizer is None:
1529         return True
1530     try:
1531         rec = KaldiRecognizer(_vosk_model, LOCAL_SAMPLE_RATE,
1532                               json.dumps(["pepper", "gaze", "[unk]"]))
1533         with wave.open(wav_path, "rb") as wf:
1534             while True:
1535                 data = wf.readframes(4000)
1536                 if len(data) == 0:
1537                     break
1538                 rec.AcceptWaveform(data)
1539                 final = json.loads(rec.FinalResult())
1540                 text = (final.get("text") or "").lower()
1541                 return "pepper" in text or "gaze" in text
1542     except Exception as e:
1543         print(f" Vosk wake-word check failed ({e}); falling through to Whisper")
1544         return True
1545
1546 #open ai whisperings and incase fails and in silent
1547 def transcribe(bypass_wake_word: bool = False,
1548               whisper_prompt: str = "User answers a quiz or chats with a companion robot.",
1549               record_again=None,
1550               max_hallucination_retries=None) -> str:
1551     # Silero VAD -> wake-word -> Whisper -> hallucination blacklist
1552     attempts_remaining = max_hallucination_retries
1553     while True:
1554         if INPUT_IS_LOCAL and not _local_speech_detected:
1555             return ""
1556
1557     # Silero VAD hard gate; NAO + LOCAL
1558     if not has_real_speech(LOCAL_WAV):

```

```

1558     print(" Silero VAD found no speech; skipping Whisper.")
1559     return ""
1560
1561     # Vosk wake-word gate; bypass for name prompt
1562     if not bypass_wake_word and not has_wake_word(LOCAL_WAV):
1563         print(" No wake-word detected; skipping Whisper.")
1564         return ""
1565
1566     try:
1567         with open(LOCAL_WAV, "rb") as fh:
1568             resp = client.audio.transcriptions.create(
1569                 model="whisper-1", #
1570                 file=fh, #
1571                 response_format="verbose_json", # get self-signals for hallucination detection
1572                 temperature=0.0, # zero randomness
1573                 prompt=whisper_prompt,
1574                 timeout=API_TIMEOUT,
1575             )
1576             text = (getattr(resp, "text", "") or "").strip()
1577             print(f" Whisper raw text: {text!r}")
1578
1579             # Whisper self-signals from verbose_json
1580             segments = getattr(resp, "segments", None) or []
1581             suspected_hallucination = False
1582             if segments:
1583                 no_speech_vals = [s.no_speech_prob for s in segments
1584                                 if getattr(s, "no_speech_prob", None) is not None]
1585                 logprob_vals = [s.avg_logprob for s in segments
1586                               if getattr(s, "avg_logprob", None) is not None]
1587                 compression_vals = [s.compression_ratio for s in segments
1588                                   if getattr(s, "compression_ratio", None) is not None]
1589                 print(f" Whisper diagnostics: no_speech={no_speech_vals},
1590                       ↪ avg_logprob={logprob_vals}, compression={compression_vals}")
1591                 if no_speech_vals and max(no_speech_vals) > 0.6:
1592                     print(f" Whisper flagged silence (max no_speech_prob={max(no_speech_vals):.2f});
1593                           ↪ dropping {text!r}")
1594                     return ""
1595                 if logprob_vals and min(logprob_vals) < -1.3:
1596                     print(f" Whisper low-confidence (min avg_logprob={min(logprob_vals):.2f});
1597                           ↪ dropping {text!r}")
1598                     return ""
1599                 # repetition-loop hallucination; flag for retry path
1600                 if compression_vals and max(compression_vals) > 2.4:
1601                     print(f" Whisper repetition loop (max
1602                           ↪ compression_ratio={max(compression_vals):.2f}); flagging {text!r} as
1603                           ↪ hallucination")
1604                     suspected_hallucination = True
1605
1606             # normalised hallucination blacklist + Whisper-self-signal hallucinations
1607             if suspected_hallucination or is_known_hallucination(text):
1608                 print(f" Filtered Whisper hallucination: {text!r}")
1609                 if record_again is not None and (attempts_remaining is None or attempts_remaining >
1610                    ↪ 0):
1611                     if attempts_remaining is not None: # whilst more retries remain decrement each
1612                         ↪ time-taken and re-record
1613                         attempts_remaining -= 1
1614                     print(f" Disregarding hallucination; listening again (attempts left:
1615                           ↪ {attempts_remaining}).")
1616             else:

```



```

1609         print(f" Disregarding hallucination; listening again.")
1610         record_again()
1611         continue
1612         return ""
1613
1614         # Strip leading "Pepper"/"Gaze" so handlers receive just the answer; \b blocks
1615         ↳ "Pepperoni"/"Gazebo" false positives..
1616         stripped = re.sub(r'(?i)^\s*(pepper|gaze)\b[.\s]*', '', text).strip()
1617         return stripped
1618     except Exception as e:
1619         print(f" Whisper transcribe failed ({e}); returning empty")
1620         return ""
1621
1622 # facial-expression pipeline
1623
1624 def capture_thread_loop(camera):
1625     """Tight capture loop; just grab the latest frame.
1626     Decoupled from detection so the dashboard runs at native fps."""
1627     global _preview_frame
1628     while True:
1629         ret, frame = camera.read()
1630         if not ret: # if camera read fails keep old frame as preview
1631             time.sleep(0.03)
1632             continue
1633         with _preview_lock:
1634             _preview_frame = frame
1635
1636 def detect_thread_loop(face_model, face_cascade):
1637     """Run cascade + CNN on a downscaled copy of the latest frame; update cached bbox + emotion.
1638     6-7 Hz is enough for a turn-based consumer."""
1639     global _last_emotion, _last_bbox
1640     while True:
1641         time.sleep(DETECT_INTERVAL_SECS)
1642         with _preview_lock:
1643             frame = _preview_frame
1644             if frame is None:
1645                 continue
1646
1647         # 2x downscale; 4x faster cascade
1648         small = cv2.resize(frame, (0, 0), fx=0.5, fy=0.5)
1649         gray = cv2.cvtColor(small, cv2.COLOR_BGR2GRAY)
1650         faces = face_cascade.detectMultiScale(gray, 1.3, 5)
1651
1652         if len(faces) == 0:
1653             bbox = None
1654             emotion, conf = "Neutral", 0.0
1655         else:
1656             x, y, w, h = max(faces, key=lambda f: f[2] * f[3])
1657             roi = gray[y:y+h, x:x+w]
1658             resized = cv2.resize(roi, (48, 48))
1659             inp = resized[np.newaxis, :, :, np.newaxis]
1660             emotion, conf = face_model.predict(inp)
1661             # scale 'bbox' back to raw-frame coordinates (was shrunk 2x)
1662             bbox = (x*2, y*2, w*2, h*2)
1663
1664         with _preview_lock:
1665             _preview_state["emotion"] = emotion
1666             _preview_state["confidence"] = conf
1667             _last_bbox = bbox

```

```

1667         _last_emotion = emotion
1668
1669 def start_preview_thread(camera, face_model, face_cascade):
1670     "Start both the capture and the detection daemon threads."
1671     cap_t = threading.Thread(target=capture_thread_loop,
1672                             args=(camera,),
1673                             daemon=True)
1674     det_t = threading.Thread(target=detect_thread_loop,
1675                             args=(face_model, face_cascade),
1676                             daemon=True)
1677     cap_t.start()
1678     det_t.start()
1679     return cap_t, det_t
1680
1681 def capture_and_classify(ssh, face_model, face_cascade,
1682                         local_camera=None) -> tuple[str, float]:
1683     "Return (emotion, confidence). Local-camera branch reads + classifies inline; NAO branch reads
1684     ↪ cached state set by detect_thread_loop off the live Pepper video stream."
1685     # local-camera + preview thread already running: just read the cache
1686     if DEBUG_PREVIEW and local_camera is not None:
1687         with _preview_lock:
1688             return _preview_state["emotion"], _preview_state["confidence"]
1689     # NAO: read cached state from detect thread
1690     if local_camera is None:
1691         with _preview_lock:
1692             return _preview_state["emotion"], _preview_state["confidence"]
1693
1694     # local-camera per-turn classification (DEBUG_PREVIEW disabled)
1695     ret, frame = local_camera.read()
1696     if not ret:
1697         print(" [Local camera failed] cv2.VideoCapture.read() returned False")
1698         return "Neutral", 0.0
1699
1700     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
1701     faces = face_cascade.detectMultiScale(gray, 1.3, 5)
1702
1703     if len(faces) == 0:
1704         bbox = None
1705         emotion, conf = "Neutral", 0.0
1706     else:
1707         x, y, w, h = max(faces, key=lambda f: f[2] * f[3])
1708         roi = gray[y:y+h, x:x+w]
1709         resized = cv2.resize(roi, (48, 48))
1710         inp = resized[np.newaxis, :, :, np.newaxis]
1711         emotion, conf = face_model.predict(inp)
1712         bbox = (x, y, w, h)
1713
1714     global _preview_frame, _last_bbox
1715     with _preview_lock:
1716         _preview_state["emotion"] = emotion
1717         _preview_state["confidence"] = conf
1718         _last_bbox = bbox
1719         _preview_frame = frame
1720
1721     return emotion, conf
1722
1723
1724 API_TIMEOUT = 10 # 10-second timeout; prevents Pepper-freeze if OpenAI/network stalls

```

```

1725
1726 def check_answer(user_answer: str, correct_answer: str,
1727                  question_context: str) -> bool:
1728     "Verify the user's answer via GPT."
1729     if not user_answer.strip():
1730         return False
1731
1732     try:
1733         resp = client.chat.completions.create(
1734             model="gpt-4.1",
1735             messages=[{
1736                 "role": "system",
1737                 "content": "You are an answer checker. Given a question, the correct answer, and the
                            ↳ user's spoken answer, determine if the user is correct. Be lenient with
                            ↳ pronunciation, phrasing, and partial answers that demonstrate knowledge.
                            ↳ Respond with ONLY 'correct' or 'incorrect'.",
1738             }, {
1739                 "role": "user",
1740                 "content": f""Question: {question_context}
                            Correct answer: {correct_answer}
                            User's answer: {user_answer}""",
1741             }],
1742             temperature=0.0,
1743             timeout=API_TIMEOUT,
1744         )
1745         verdict = resp.choices[0].message.content.strip().lower()
1746         return verdict == "correct" # if AI says its 'correct' return verdict=correct
1747     except Exception as e:
1748         print(f" Answer verifier API failed: {e}")
1749         fallback_answer = correct_answer.lower().strip()
1750         fallback_user = user_answer.lower().strip()
1751
1752         return fallback_user == fallback_answer
1753
1754 # save/load sessions
1755
1756 def save_session(engine: AdaptiveEngine, preferred_game: Optional[GameType] = None,
1757                  quiet: bool = False):
1758     """Save session progress so user can continue later.
1759     quiet=True suppresses the "Session saved to ..." print, used when saving
1760     after every round so the log doesn't fill up with save confirmations.
1761     """
1762     data = {
1763         "total_correct": engine.total_correct,
1764         "total_rounds_played": engine.total_rounds_played,
1765         "best_streak": engine.best_streak,
1766         "games_played": {g.value: c for g, c in engine.games_played.items()},
1767         "game_switches": engine.game_switch_count,
1768         "last_difficulty": engine.current_difficulty.value,
1769         "last_game": engine.current_game.value,
1770         "preferred_game": preferred_game.value if preferred_game else None,
1771         "rounds_played": len(engine.history),
1772         "recent_questions": engine.recent_questions[-30:],
1773         "recent_answers": engine.recent_answers[-30:],
1774         "recent_game_types": engine.recent_game_types[-30:],
1775         "round_log": [
1776             {
1777                 "round": r.round_number,
1778                 "game": r.game_type.value,
1779             }
1780         ]

```

```

1781         "difficulty": r.difficulty.value,
1782         "correct": r.correct,
1783         "time": round(r.response_time, 1),
1784         "expression": r.facial_expression,
1785         "vocal": r.vocal_emotion,
1786         "state": r.inferred_state.value,
1787     }
1788     for r in engine.history
1789 ],
1790 }
1791 with open(SAVE_FILE, "w") as f:
1792     json.dump(data, f, indent=2)
1793 if not quiet:
1794     print(f" Session saved to {SAVE_FILE}")
1795
1796 def load_session() -> Optional[dict]:
1797     "Load a previously saved session, if one exists."
1798     if not os.path.exists(SAVE_FILE):
1799         return None
1800     try:
1801         with open(SAVE_FILE, "r") as f:
1802             return json.load(f)
1803     except (json.JSONDecodeError, IOError) as e:
1804         print(f" Save file corrupt, ignoring: {e}")
1805         return None
1806
1807 def restore_engine(save_data: dict) -> AdaptiveEngine:
1808     "Restore engine state from saved data."
1809     engine = AdaptiveEngine()
1810     engine.total_correct = save_data.get("total_correct", 0)
1811     # legacy fallback; older saves used rounds_played instead
1812     engine.total_rounds_played = save_data.get("total_rounds_played",
1813                                              save_data.get("rounds_played", 0))
1814     engine.best_streak = save_data.get("best_streak", 0)
1815     engine.game_switch_count = save_data.get("game_switches", 0)
1816     engine.current_difficulty = Difficulty(save_data.get("last_difficulty", 2))
1817     engine.current_game = GameType(save_data.get("last_game", "numbers"))
1818     engine.recent_questions = list(save_data.get("recent_questions", []))[-30:]
1819     engine.recent_answers = list(save_data.get("recent_answers", []))[-30:]
1820     engine.recent_game_types = list(save_data.get("recent_game_types", []))[-30:]
1821     for g_val, count in save_data.get("games_played", {}).items():
1822         try:
1823             engine.games_played[GameType(g_val)] = count
1824         except ValueError as e:
1825             print(f" Could not restore game type {g_val!r}: {e}")
1826     return engine
1827
1828 def delete_save():
1829     "Remove the save file after a completed session or on user request."
1830     if os.path.exists(SAVE_FILE):
1831         os.remove(SAVE_FILE)
1832
1833 # OpenAI function-calling + conversation helpers
1834 '''
1835 - generate_game_question: called by the LLM when it wants to ask a new question; returns a
1836     ↳ question + correct answer for the specified game type and difficulty
1837 - check_game_answer: called by the LLM after the user answers a question; returns answer
1838     ↳ correctness if so

```

```

1838     - evaluate_last_adaptation: called by the LLM to self-evaluate if the previous round's
1839       ↪ adaptive strategy did indeed help the user
1840
1841     - request_more_time: called by the LLM when the user asks for more time
1842
1843     '''
1844     TOOLS = [
1845         {
1846             "type": "function",
1847             "function": {
1848                 "name": "generate_game_question",
1849                 "description": "Generate a new countdown-style game question. Call this when the
1850                   ↪ conversation naturally leads to playing a game.",
1851                 "parameters": {
1852                     "type": "object",
1853                     "properties": {
1854                         "game_type": {
1855                             "type": "string",
1856                             "enum": ["numbers", "letters"],
1857                             "description": "Type of countdown game round.",
1858                         },
1859                         "difficulty": {
1860                             "type": "string",
1861                             "enum": ["EASY", "MEDIUM", "HARD"],
1862                             "description": "Difficulty level for the question.",
1863                         },
1864                     },
1865                     "required": ["game_type", "difficulty"],
1866                 },
1867             },
1868         },
1869         {
1870             "type": "function",
1871             "function": {
1872                 "name": "check_game_answer",
1873                 "description": "Verify whether the user's spoken answer to the current game question is
1874                   ↪ correct. Call this after the user gives an answer to an active game question.",
1875                 "parameters": {
1876                     "type": "object",
1877                     "properties": {
1878                         "user_answer": {
1879                             "type": "string",
1880                             "description": "What the user said.",
1881                         },
1882                         "correct_answer": {
1883                             "type": "string",
1884                             "description": "The known correct answer.",
1885                         },
1886                         "question_context": {
1887                             "type": "string",
1888                             "description": "The original question text for context.",
1889                         },
1890                     },
1891                     "required": ["user_answer", "correct_answer", "question_context"],
1892                 },
1893             },
1894         },
1895     ]

```

```

1894         "name": "evaluate_last_adaptation",
1895         "description": "Self-evaluate whether the previous round's adaptive strategy actually
                        ↳ helped the user. Returns a natural-language evaluation.",
1896         "parameters": {"type": "object", "properties": {}},
1897     },
1898 },
1899 {
1900     "type": "function",
1901     "function": {
1902         "name": "request_more_time",
1903         "description": "The user has asked for more time to think about the current game
                        ↳ question. Acknowledge their request warmly.",
1904         "parameters": {"type": "object", "properties": {}},
1905     },
1906 },
1907 ]
1908
1909 def build_signal_context(engine: AdaptiveEngine,
1910                        expression: str, expr_conf: float,
1911                        vocal_emo: str, vocal_conf: float,
1912                        vol_rms: float, response_time: float) -> str:
1913     "Build a signal summary string injected alongside every user message so the LLM can adapt its
        ↳ behaviour to the user's real-time emotional state without explicit adaptive-engine
        ↳ instructions."
1914     correctness = engine.rolling_correctness()
1915     recent_faces = [r.facial_expression for r in engine.history[-3:]]
1916     recent_vocal = [r.vocal_emotion for r in engine.history[-3:]]
1917
1918     # map raw budget to a semantic label so the LLM reflects pacing without ever seeing or
        ↳ repeating the raw seconds verbatim in dialogue
1919     if engine.think_budget_secs >= 17.0:
1920         pacing = "relaxed and patient"
1921     elif engine.think_budget_secs <= 13.0:
1922         pacing = "brisk and energetic"
1923     else:
1924         pacing = "standard"
1925
1926     lines = [
1927         "--- LIVE SIGNALS ---",
1928         f"Turn: {engine.round_number}",
1929         f"Face: {expression} ({expr_conf:.0%}) [PRIMARY signal: trust this first]",
1930         f"Voice: {vocal_emo} ({vocal_conf:.0%}) [advisory only; the vocal model is noisy and often
            ↳ stuck on 'fearful']",
1931         f"Volume: {vol_rms:.0f} RMS",
1932         f"Response time: {response_time:.1f}s",
1933         f"Rolling accuracy (last {CORRECTNESS_WINDOW}): {correctness:.0%}", # overall correctness
1934         f"System pacing: {pacing}",
1935     ]
1936     if recent_faces:
1937         lines.append(f"Recent faces: {'', '.join(recent_faces)}")
1938     if recent_vocal:
1939         lines.append(f"Recent vocal: {'', '.join(recent_vocal)}")
1940
1941     return "\n".join(lines)
1942
1943 def converse(conversation: list, tools: list) -> object:
1944     "General-conversation OpenAI `gpt-5.4` chat-completion call with function calling."
1945     try:
1946         resp = client.chat.completions.create(

```

```

1947         model="gpt-5.4",
1948         messages=conversation,
1949         tools=tools,
1950         temperature=0.8, # bit of randomness/creativity for more interesting companion
1951         timeout=API_TIMEOUT,
1952     )
1953     return resp.choices[0].message
1954 except Exception as e:
1955     print(f"converse() API failed... : {e}")
1956     return types.SimpleNamespace(
1957         content="I had a brief network hiccup. Let's keep going! [gesture:think]",
1958         tool_calls=None, role="assistant")
1959
1960 def execute_tool_call(tool_name: str, tool_args: dict,
1961                       engine: AdaptiveEngine, game_state: GameState,
1962                       conversation: list,
1963                       preferred_game: Optional[GameType],
1964                       dashboard=None) -> str:
1965     "Dispatch a function-calling tool invocation and return a JSON string result."
1966     if tool_name == "generate_game_question":
1967         gt = tool_args.get("game_type", "numbers")
1968         diff = tool_args.get("difficulty", "MEDIUM")
1969         result = generate_game_question_internal(
1970             gt, diff,
1971             recent=engine.recent_questions,
1972             recent_answers=engine.recent_answers,
1973             recent_game_types=engine.recent_game_types,
1974         )
1975         # Record question, answer, AND game_type so the next generation call sees the full
1976         ↪ do-not-repeat context; answer-level dedup catches mode-collapsed targets even when
1977         ↪ GPT rephrases the question; game-type history pushes rotation so we don't get 5
1978         ↪ Numbers games in a row
1979         new_q = result.get("question", "").strip()
1980         new_a = str(result.get("answer", "")).strip()
1981         if new_q:
1982             engine.recent_questions.append(new_q)
1983             engine.recent_questions = engine.recent_questions[-30:]
1984         if new_a:
1985             engine.recent_answers.append(new_a)
1986             engine.recent_answers = engine.recent_answers[-30:]
1987         engine.recent_game_types.append(gt)
1988         engine.recent_game_types = engine.recent_game_types[-30:]
1989         # Sync adaptive engine with LLM's chosen difficulty so the engine's next decide() starts
1990         ↪ from the correct baseline
1991         try:
1992             engine.current_difficulty = Difficulty[diff]
1993         except (KeyError, ValueError):
1994             pass
1995         # also sync current_game; record_round logs game_type from this
1996         try:
1997             engine.current_game = GameType(gt)
1998         except ValueError:
1999             pass
2000         game_state.active = True
2001         game_state.current_question = result.get("question", "")
2002         game_state.current_answer = result.get("answer", "")
2003         game_state.category = result.get("category", "")
2004         return json.dumps(result)

```

```

2002     elif tool_name == "check_game_answer":
2003         ua = tool_args.get("user_answer", "")
2004         if not game_state.active:
2005             game_state.last_answer_checked = False
2006             return json.dumps({"correct": False, "error": "no active game question"})
2007             # snapshot before same-chain regen overwrites
2008             game_state.answered_question = game_state.current_question
2009             game_state.answered_answer = game_state.current_answer
2010             game_state.answered_game_type = engine.current_game
2011             game_state.answered_difficulty = engine.current_difficulty
2012             # Python state is the source of truth, not LLM-supplied args
2013             is_correct = check_answer(
2014                 ua,
2015                 game_state.current_answer,
2016                 game_state.current_question,
2017             )
2018             # last_answer_checked: ensures record_round() runs
2019             game_state.last_answer_checked = True
2020             game_state.last_answer_correct = is_correct
2021             if is_correct:
2022                 game_state.active = False
2023             return json.dumps({"correct": is_correct})
2024
2025     elif tool_name == "evaluate_last_adaptation":
2026         evaluation = engine.evaluate_adaptation()
2027         return json.dumps({"evaluation": evaluation})
2028
2029     elif tool_name == "request_more_time":
2030         game_state.waiting = True
2031         return json.dumps({"acknowledged": True})
2032
2033     else:
2034         return json.dumps({"error": f"Unknown tool: {tool_name}"})
2035
2036 def process_llm_response(message, conversation: list,
2037                          engine: AdaptiveEngine, game_state: GameState,
2038                          preferred_game: Optional[GameType],
2039                          dashboard=None) -> str:
2040     "Handle the LLM response, including any tool call chains."
2041     msg_dict = {"role": "assistant", "content": message.content or ""}
2042     if message.tool_calls: # if tool calls are required inject the function-call into convo
2043         ↳ history so LLM can decide persistence
2044         msg_dict["tool_calls"] = [
2045             {
2046                 "id": tc.id,
2047                 "type": "function",
2048                 "function": {"name": tc.function.name, "arguments": tc.function.arguments},
2049             }
2050             for tc in message.tool_calls # iterate over each tool call in the message
2051         ]
2052     conversation.append(msg_dict)
2053     # Cap recursive tool calls at 5 rounds to prevent infinite loops
2054     max_tool_rounds = 5
2055     current_msg = message
2056     used_answer_check = False # gate same-chain regen so engine.decide can adapt difficulty first
2057     for _ in range(max_tool_rounds):
2058         if not current_msg.tool_calls:
2059             break # kill loop if no more tool calls

```



```

2060     # block same-batch regen; engine.decide() must update difficulty first
2061     batch_has_check = any(tc.function.name == "check_game_answer"
2062                           for tc in current_msg.tool_calls)
2063     for tc in current_msg.tool_calls:
2064         fn_name = tc.function.name
2065         try:
2066             fn_args = json.loads(tc.function.arguments)
2067         except json.JSONDecodeError:
2068             fn_args = {}
2069
2070         if batch_has_check and fn_name == "generate_game_question":
2071             result_str = json.dumps({"skipped": True,
2072                                     "reason": "deferred until after adaptive decision"})
2073             print(f" Skipping {fn_name}: deferred after answer check")
2074         else:
2075             print(f" Calling tool {fn_name}({fn_args})")
2076             result_str = execute_tool_call(
2077                 fn_name, fn_args, engine, game_state,
2078                 conversation, preferred_game, dashboard
2079             )
2080             print(f" Tool returned: {result_str[:120]}")
2081         if fn_name == "check_game_answer":
2082             used_answer_check = True
2083             conversation.append({
2084                 "role": "tool",
2085                 "tool_call_id": tc.id,
2086                 "content": result_str,
2087             })
2088         # withhold generate_game_question after a check; defer to next turn so engine.decide can
2089         #   ↪ update difficulty first
2090     next_tools = TOOLS
2091     current_msg = converse(conversation, next_tools)
2092     resp_dict = {"role": "assistant", "content": current_msg.content or ""}
2093     if current_msg.tool_calls:
2094         resp_dict["tool_calls"] = [
2095             {
2096                 "id": tc.id,
2097                 "type": "function",
2098                 "function": {"name": tc.function.name, "arguments": tc.function.arguments},
2099             }
2100             for tc in current_msg.tool_calls
2101         ]
2102     conversation.append(resp_dict)
2103
2104     return current_msg.content or ""
2105
2106 #look for gestures
2107 def extract_gesture(text: str) -> str:
2108     "Returns wave always -only gesture now used."
2109     return "wave"
2110
2111 def validate_numbers_round(numbers: list, target: int) -> bool:
2112     """"Skipped -too slow for real-time use. GPT is instructed to verify its own answer.""
2113     return True
2114
2115 def generate_game_question_internal(game_type_str: str, difficulty_str: str,
2116                                    recent: Optional[list[str]] = None,
2117                                    recent_answers: Optional[list[str]] = None,
2118                                    recent_game_types: Optional[list[str]] = None) -> dict:

```

```

2118     try:
2119         gt = GameType(game_type_str)
2120     except ValueError:
2121         gt = GameType.NUMBERS
2122     try:
2123         diff = Difficulty[difficulty_str]
2124     except (KeyError, ValueError):
2125         diff = Difficulty.MEDIUM
2126
2127     import secrets
2128     variety_seed = secrets.token_hex(3)
2129
2130     banned_questions = "\n".join(f"- {q}" for q in (recent or [])[-30:])
2131     banned_answers = ", ".join((recent_answers or [])[-30:])
2132
2133     prompt = f"""You are generating a {game_type_str.upper()} round for a Countdown-style game.
2134
2135     GAME TYPE: {game_type_str.upper()} -do not generate any other type.
2136     DIFFICULTY: {DIFFICULTY_DESCRIPTIONS[diff]}
2137     VARIETY SEED (use this to pick different numbers/letters every time): {variety_seed}
2138
2139     STRICT RULES -you MUST follow all of these:
2140     1. This MUST be a {game_type_str.upper()} round. No exceptions.
2141     2. The question MUST be completely different from every question in the BANNED LIST below.
2142     3. The correct answer MUST NOT appear in the BANNED ANSWERS list below.
2143     4. Pick a fresh number set or letter set -do not reuse combinations you have used before.
2144
2145     BANNED LIST (do not repeat or closely mimic any of these):
2146     {banned_questions if banned_questions else "none yet"}
2147
2148     BANNED ANSWERS (your answer must not be any of these):
2149     {banned_answers if banned_answers else "none yet"}
2150
2151     Respond with a JSON object only -no markdown, no code fences:
2152     {{ "question": "...", "answer": "...", "category": "..." }}
2153 """"
2154
2155     try:
2156         resp = client.chat.completions.create(
2157             model="gpt-5.4",
2158             messages=[
2159                 {"role": "system", "content": (
2160                     "You generate countdown-style game questions. Respond ONLY with valid JSON. "
2161                     "Follow all rules in the user prompt exactly. "
2162                     "For numbers rounds you MUST verify your arithmetic before responding. "
2163                     "The 'answer' field must contain ONLY the final solution expression -"
2164                     "for example '50 + 25 = 75'. "
2165                     "Do NOT include working out, alternative attempts, explanations, "
2166                     "or commentary in the answer field. One clean expression only."
2167                 )},
2168                 {"role": "user", "content": prompt},
2169             ],
2170             temperature=1.2,
2171             timeout=API_TIMEOUT,
2172         )
2173
2174     content = resp.choices[0].message.content.strip()
2175     if content.startswith("```"):
2176         content = content.split("\n", 1)[1] if "\n" in content else content[3:]

```

```

2177         if content.endswith("`"):
2178             content = content[:-3]
2179             content = content.strip()
2180         result = json.loads(content)
2181
2182         # validate numbers rounds as GPT regularly hallucinates unreachable targets
2183         if game_type_str == "numbers":
2184             try:
2185                 import re
2186                 q = result.get("question", "")
2187                 nums = list(map(int, re.findall(r'\b\d+\b', q.split("make")[0])))
2188                 target_match = re.search(r'make.*?(\d+)', q)
2189                 if target_match and nums:
2190                     target = int(target_match.group(1))
2191                     if not validate_numbers_round(nums, target):
2192                         print(f" GPT produced unreachable target {target} from {nums}; using
2193                             ↪ fallback")
2194                         return {
2195                             "question": "Using 25, 50, 3, 6, 4, and 2, make the target 100.",
2196                             "answer": "25 x 4 = 100",
2197                             "category": "numbers fallback",
2198                         }
2199             except Exception as ve:
2200                 print(f" Numbers validation skipped: {ve}")
2201
2202         return result
2203     except json.JSONDecodeError:
2204         return {"question": content if 'content' in locals() else "", "answer": "", "category":
2205             ↪ "general"}
2206     except Exception as e:
2207         print(f" Game question gen failed: {e}")
2208         return {
2209             "question": "Let's try a quick one: what is 25 + 17?",
2210             "answer": "42",
2211             "category": "arithmetic fallback",
2212         }
2213
2214     _STATE_COLOURS = {
2215         InferredState.THRIVING: "green",
2216         InferredState.COMFORTABLE: "blue",
2217         InferredState.STRUGGLING: "orange",
2218         InferredState.FRUSTRATED: "red",
2219         InferredState.DISENGAGED: "grey",
2220     }
2221
2222     class GazeDashboard:
2223         """Tkinter dashboard for GAZE.
2224         1- create a window with two panels: left = conversation then video, right for inferred
2225             ↪ stats/signals/state
2226         2- left panel: camera canvas above the scrollable conversation log; log is read-only default
2227             ↪ and
2228             flipped to "normal" when a new line is appended
2229         3- right panel: round/score/streak counters at top, then the transcription block (what the
2230             ↪ user said + correct/incorrect,
2231             recoloured per outcome), then the live signals (face CNN, voice MLP, volume RMS, response
2232             ↪ time, rolling accuracy,
2233             think budget), then the adaptive decision block (inferred state, difficulty, tone,
2234             ↪ adaptations, plus a grey eval-note line below)
2235         4- quit handling: the Quit button, Esc, Cmd/Ctrl-Q and the window-close (X) all converge on

```

```

    ↪ quit_app(), so the SSH farewell + LED-off
2229 always run no matter how the user closes the dashboard
2230 5- refresh loops: camera_refresh() composites the latest raw frame with the cached detection
    ↪ overlay at native fps; signal_refresh()
2231 repaints the StringVar-bound labels at a slower cadence; both self-reschedule via
    ↪ root.after()
2232 """
2233
2234 CAMERA_W, CAMERA_H = 400, 300
2235
2236 def __init__(self, ssh=None, ssh_tts=None):
2237     # stashed for quit_app farewell + LED-off
2238     self._ssh = ssh
2239     self._ssh_tts = ssh_tts
2240     self.root = tk.Tk()
2241     self.root.title("GAZE Dashboard")
2242     self.root.resizable(False, False)
2243
2244     # left col: video + chat; right col: stats
2245     left = tk.Frame(self.root)
2246     left.grid(row=0, column=0, padx=8, pady=8, sticky="n")
2247
2248     # camera canvas; black letterboxes empty pixels
2249     self.camera_label = tk.Label(left, bg="black",
2250                                 width=self.CAMERA_W, height=self.CAMERA_H)
2251     self.camera_label.pack()
2252
2253     tk.Label(left, text="Conversation:").pack(anchor="w", pady=(6, 0))
2254     conv_frame = tk.Frame(left)
2255     conv_frame.pack()
2256     # read-only by default; helpers flip to insert
2257     self._conv_text = tk.Text(conv_frame, font=("Courier", 10),
2258                               width=50, height=10, wrap="word",
2259                               state="disabled")
2260     # scrollbar bidirectional wiring
2261     conv_scroll = tk.Scrollbar(conv_frame, command=self._conv_text.yview)
2262     self._conv_text.configure(yscrollcommand=conv_scroll.set)
2263     self._conv_text.pack(side="left", fill="both", expand=True)
2264     conv_scroll.pack(side="right", fill="y")
2265
2266     right = tk.Frame(self.root)
2267     right.grid(row=0, column=1, padx=(0, 8), pady=8, sticky="n")
2268
2269     tk.Label(right, text="GAZE (Game-Adaptive Zone of Engagement) Dashboard",
2270             font=("TkDefaultFont", 14, "bold")).pack(pady=(0, 4))
2271
2272     self._round_var = tk.StringVar(value="Round: -")
2273     self._score_var = tk.StringVar(value="Score: 0/0")
2274     self._streak_var = tk.StringVar(value="Streak: 0")
2275     for var in (self._round_var, self._score_var, self._streak_var):
2276         tk.Label(right, textvariable=var).pack(anchor="w")
2277
2278     # user transcription only; answer on stdout
2279     tk.Label(right, text="").pack()
2280     tk.Label(right, text="Transcription:",
2281             font=("TkDefaultFont", 10, "bold")).pack(anchor="w")
2282     self._heard_var = tk.StringVar(value="You said: -")
2283     self._result_var = tk.StringVar(value="")
2284     tk.Label(right, textvariable=self._heard_var).pack(anchor="w")

```

```

2285     # fg recolour on correctness
2286     self._result_label = tk.Label(right, textvariable=self._result_var,
2287                                   font=("TkDefaultFont", 11, "bold"))
2288     self._result_label.pack(anchor="w")
2289
2290     tk.Label(right, text="").pack()
2291     tk.Label(right, text="Live Signals:",
2292              font=("TkDefaultFont", 10, "bold")).pack(anchor="w")
2293     self._face_var = tk.StringVar(value="Face (CNN): -")
2294     self._voice_var = tk.StringVar(value="Voice (MLP): -")
2295     self._vol_var = tk.StringVar(value="Volume RMS: -")
2296     self._time_var = tk.StringVar(value="Response time: -")
2297     self._acc_var = tk.StringVar(value="Rolling acc: -")
2298     self._budget_var = tk.StringVar(value="Think budget: -")
2299     for var in (self._face_var, self._voice_var, self._vol_var,
2300               self._time_var, self._acc_var, self._budget_var):
2301         tk.Label(right, textvariable=var,
2302                  font=("Courier", 10)).pack(anchor="w")
2303
2304     tk.Label(right, text="").pack()
2305     tk.Label(right, text="Adaptive Decision:",
2306              font=("TkDefaultFont", 10, "bold")).pack(anchor="w")
2307     self._state_var = tk.StringVar(value="State: -")
2308     # kept for state-coloured fg
2309     self._state_label = tk.Label(right, textvariable=self._state_var,
2310                                  font=("TkDefaultFont", 11, "bold"))
2311     self._state_label.pack(anchor="w")
2312
2313     self._diff_var = tk.StringVar(value="Difficulty: -")
2314     self._tone_var = tk.StringVar(value="Tone: -")
2315     self._adapt_var = tk.StringVar(value="Adaptations: -")
2316     for var in (self._diff_var, self._tone_var, self._adapt_var):
2317         tk.Label(right, textvariable=var).pack(anchor="w")
2318
2319     # adaptation-eval note (miniscule 'grey' text)
2320     self._eval_var = tk.StringVar(value="")
2321     self._eval_label = tk.Label(right, textvariable=self._eval_var,
2322                                  font=("TkDefaultFont", 9), fg="grey",
2323                                  wraplength=340, justify="left")
2324     self._eval_label.pack(anchor="w")
2325
2326     tk.Button(right, text="Quit (Esc)",
2327              command=self.quit_app).pack(pady=(10, 0))
2328
2329     self.root.bind("<Escape>", lambda e: self.quit_app())
2330     # Cmd-Q is macOS-only; guard each
2331     for combo in ("<Command-q>", "<Control-q>"):
2332         try:
2333             self.root.bind(combo, lambda e: self.quit_app())
2334         except tk.TclError:
2335             pass
2336     # route window-close through quit_app
2337     self.root.protocol("WM_DELETE_WINDOW", self.quit_app)
2338
2339     self.camera_refresh()
2340     self.signal_refresh()
2341     self.root.update() # paint before mainloop()
2342
2343

```

```

2344 def camera_refresh(self):
2345     """Composite the latest raw frame with the cached detection overlay.
2346     Bbox is drawn on every fresh raw frame for a live-feed."""
2347     with _preview_lock:
2348         frame = _preview_frame
2349         bbox = _last_bbox
2350         emotion = _preview_state["emotion"]
2351         conf = _preview_state["confidence"]
2352
2353     if frame is not None:
2354         display = frame.copy() # don't mutate the shared raw frame; the
2355             ↳ bbox/text overlays are per-paint
2356         if bbox is not None:
2357             x, y, w, h = bbox
2358             cv2.rectangle(display, (x, y), (x + w, y + h), (0, 255, 0), 2)
2359             label = f"{emotion} ({conf:.0%})"
2360             cv2.putText(display, label, (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
2361                 0.8, (0, 255, 0), 2)
2362             rgb = cv2.cvtColor(display, cv2.COLOR_BGR2RGB) # OpenCV is BGR, Tk/PIL expect RGB; swap
2363                 ↳ channels here
2364             small = cv2.resize(rgb, (self.CAMERA_W, self.CAMERA_H))
2365             img = ImageTk.PhotoImage(Image.fromarray(small)) # Tk-compatible image wrapper around
2366                 ↳ the PIL image
2367             self.camera_label.configure(image=img)
2368             self.camera_label._photo = img # prevent garbage collection -Tk doesn't keep a ref to
2369                 ↳ PhotoImage so without an attribute hold the image gets GC'd and the canvas blanks
2370
2371     self.root.after(33, self.camera_refresh) # ~30 fps target -Tk's standard self-rescheduling
2372         ↳ animation pattern; after(ms, fn) queues fn onto the mainloop after ms milliseconds
2373
2374 def signal_refresh(self):
2375     with _preview_lock:
2376         emotion = _preview_state["emotion"]
2377         conf = _preview_state["confidence"]
2378     self._face_var.set(f"Face (CNN): {emotion} ({conf:.0%})")
2379     self.root.after(500, self.signal_refresh) # 2 Hz
2380
2381 # thread-safe scheduling: tkinter widgets are main-thread only (macOS); root.after(0, fn)
2382     ↳ queues fn onto the main loop.
2383
2384 def on_main(self, fn):
2385     "Schedule fn to run on the main (tkinter) thread."
2386     try:
2387         self.root.after(0, fn)
2388     except tk.TclError:
2389         pass
2390
2391 def update_robot_speech(self, text: str):
2392     # enable /to insert /to see end /to disable: the Text widget is "disabled" (read-only) by
2393         ↳ default; flip to "normal" to write, scroll the view to "end" so newest message stays
2394         ↳ visible, then re-disable so the user can't accidentally type into the log.
2395     def apply():
2396         self._conv_text.configure(state="normal")
2397         self._conv_text.insert("end", f"Robot: {text}\n\n")
2398         self._conv_text.see("end")
2399         self._conv_text.configure(state="disabled")
2400     self.on_main(apply) # schedule the UI update on the main thread
2401

```

```

2395 def append_user_speech(self, text: str):
2396     def apply():
2397         self._conv_text.configure(state="normal")
2398         self._conv_text.insert("end", f"You: {text}\n")
2399         self._conv_text.see("end")
2400         self._conv_text.configure(state="disabled")
2401         self._heard_var.set(f"You said: {text}") # mirror to right panel
2402     self.on_main(apply)
2403
2404 def update_think_budget(self, secs: float):
2405     "Update the dashboard's adaptive think-budget diagnostic row."
2406     def apply():
2407         self._budget_var.set(f"Think budget: {secs:.1f}s")
2408     self.on_main(apply)
2409
2410 def update_signals(self, round_num: int, user_answer: str, correct_answer: str,
2411                   correct: bool, expression: str, expr_conf: float,
2412                   vocal_emo: str, vocal_conf: float, vol_rms: float,
2413                   response_time: float, rolling_acc: float,
2414                   total_correct: int, total_rounds: int, streak: int):
2415     def apply():
2416         self._round_var.set(f"Round: {round_num}")
2417         self._score_var.set(f"Score: {total_correct}/{total_rounds}")
2418         self._streak_var.set(f"Streak: {streak}")
2419
2420         self._heard_var.set(f"You said: {user_answer if user_answer else '(silence)'}") #
2421             ↪ extract user's answer
2422         if not correct_answer: # baseline; non-active game state
2423             self._result_var.set("")
2424             self._result_label.configure(fg="grey")
2425         elif correct:
2426             self._result_var.set("CORRECT")
2427             self._result_label.configure(fg="green")
2428         elif not user_answer:
2429             self._result_var.set("NO ANSWER")
2430             self._result_label.configure(fg="grey")
2431         else:
2432             self._result_var.set("INCORRECT")
2433             self._result_label.configure(fg="red")
2434
2435         self._face_var.set(f"Face (CNN): {expression} ({expr_conf:.0%})")
2436         self._voice_var.set(f"Voice (MLP): {vocal_emo} ({vocal_conf:.0%})")
2437
2438         vol_tag = "(loud)" if vol_rms > 2000 else "(quiet)" if vol_rms < VOLUME_THRESHOLD else
2439             ↪ "(normal)"
2440         bar_len = min(int(vol_rms / 250), 20)
2441         meter = "#" * bar_len + "." * (20 - bar_len)
2442         self._vol_var.set(f"Volume RMS: {vol_rms:>5.0f} [{meter}] {vol_tag}")
2443         self._time_var.set(f"Response time: {response_time:.1f}s")
2444         self._acc_var.set(f"Rolling acc: {rolling_acc:.0%}")
2445
2446     self.on_main(apply)
2447
2448 def update_decision(self, decision, adaptation_eval: str = None):
2449     state = decision.inferred_state
2450     def apply():
2451         self._state_var.set(f"State: {state.value.upper()}")
2452         self._state_label.configure(fg=STATE_COLOURS.get(state, "grey"))
2453         self._diff_var.set(f"Difficulty: {decision.difficulty.name}")

```

```

2452         self._tone_var.set(f"Tone: {decision.tone}")
2453         flags = []
2454         if decision.give_hint: flags.append("hint")
2455         if decision.give_encouragement: flags.append("encouragement")
2456         if decision.switch_game: flags.append(f"switch -> {decision.game_type.value}")
2457         self._adapt_var.set(f"Adaptations: {'', '.join(flags) if flags else 'none'}")
2458         self._eval_var.set(adaptation_eval if adaptation_eval else "")
2459     self.on_main(apply)
2460
2461     def quit_app(self):
2462         "Speak a farewell, dim Pepper's eye-LEDs, then kill the process."
2463         print("\n [Dashboard] Quit requested.")
2464         if not LOCAL_MODE and self._ssh is not None:
2465             nao_set_leds(self._ssh, "FaceLeds", 0x00000000, 0.5)
2466             # local_say in LOCAL_MODE; SSH-TTS otherwise
2467             say(self._ssh_tts, "It was great to chat! See you next time!")
2468             self.close()
2469             os._exit(0)
2470
2471     def close(self):
2472         try:
2473             self.root.destroy()
2474         except tk.TclError:
2475             pass
2476
2477     def is_goodbye(text: str) -> bool:
2478         "LLM goodbye-intent gate; True if the user signalled they want to leave."
2479         if not text:
2480             return False
2481         try:
2482             bye = client.chat.completions.create(
2483                 model="gpt-4.1", max_completion_tokens=5, temperature=0.0,
2484                 timeout=API_TIMEOUT,
2485                 messages=[{"role": "system", "content":
2486                     "Did the user just signal they want to end the conversation (e.g. 'bye', 'goodbye',
2487                     ↪ 'I'm done', 'see you later', 'I have to go')? Reply ONLY with 'GOODBYE' or
2488                     ↪ 'CONTINUE'."},
2489                     {"role": "user", "content": text}],
2490             ).choices[0].message.content.strip().upper()
2491         except Exception:
2492             return False
2493         return "GOODBYE" in bye
2494
2495     def conversation_loop(dashboard, face_model, face_cascade, speech_model,
2496                          local_camera, ssh, ssh_tts, energy_threshold):
2497         "Comprehensive main-conversation loop."
2498         preferred_game = None
2499         engine = AdaptiveEngine()
2500         game_state = GameState()
2501
2502         save_data = load_session()
2503         if save_data:
2504             # legacy fallback; older saves only had per-session rounds_played
2505             prev_rounds = save_data.get("total_rounds_played",
2506                                         save_data.get("rounds_played", 0))
2507             prev_correct = save_data.get("total_correct", 0)
2508
2509         welcome_back = f"Welcome back! Last time you played {prev_rounds} rounds and got

```



```

    ↪ {prev_correct} correct. Want to continue where you left off, or start fresh?"
2509 if not LOCAL_MODE:
2510     nao_track_face(ssh, enable=True)
2511     nao_set_leds(ssh, "FaceLeds", 0x0000FF00, 1.0)
2512     nao_gesture(ssh, "wave")
2513     say(ssh_tts, welcome_back)
2514     print(f"\nRobot: {welcome_back}")
2515     dashboard.update_robot_speech(welcome_back)
2516
2517     print("\nListening for continue/fresh...")
2518     if not LOCAL_MODE:
2519         nao_set_leds(ssh, "EarLeds", 0x0000FF00, 0.3)
2520     record(ssh, energy_threshold,
2521            no_speech_max=4.0, silence_secs=2.0, record_max_secs=6.0)
2522
2523     expression, expr_conf = capture_and_classify(
2524         ssh, face_model, face_cascade, local_camera)
2525     vocal_emo, vocal_conf = classify_speech_emotion(speech_model, LOCAL_WAV)
2526     vol_rms = measure_volume()
2527     dashboard.update_signals(
2528         round_num=0, user_answer="", correct_answer="", correct=False,
2529         expression=expression, expr_conf=expr_conf,
2530         vocal_emo=vocal_emo, vocal_conf=vocal_conf,
2531         vol_rms=vol_rms, response_time=0, rolling_acc=0,
2532         total_correct=0, total_rounds=0, streak=0,
2533     )
2534
2535     resume_text = transcribe(
2536         bypass_wake_word=True,
2537         record_again=lambda: record(ssh, energy_threshold,
2538                                    no_speech_max=4.0, silence_secs=2.0, record_max_secs=6.0),
2539     )
2540     if resume_text:
2541         print(f" Heard: {resume_text}")
2542         dashboard.append_user_speech(resume_text)
2543         if is_goodbye(resume_text):
2544             dashboard.quit_app()
2545     else:
2546         print(" Heard: (silence)")
2547
2548     # LLM intent classifier; on failure fall back to a keyword check, then
2549     # default to CONTINUE so an API timeout never silently nukes the save.
2550     intent = ""
2551     if resume_text:
2552         try:
2553             intent = client.chat.completions.create(
2554                 model="gpt-4.1", max_completion_tokens=5, temperature=0.0,
2555                 timeout=API_TIMEOUT,
2556                 messages=[{"role": "system", "content":
2557                     "The user was just asked whether they want to CONTINUE their previous session
2558                     ↪ or start FRESH. Reply ONLY with 'CONTINUE' or 'FRESH'."},
2559                     {"role": "user", "content": resume_text}],
2560             ).choices[0].message.content.strip().upper()
2561         except Exception as e:
2562             print(f" Resume intent classification failed ({e}); falling back to keyword match")
2563             low = resume_text.lower()
2564             fresh_kw = ("fresh", "start over", "restart", "new game", "start again", "begin
2565                 ↪ again", "from scratch")
2566             continue_kw = ("continue", "carry on", "keep going", "resume", "where i left",

```

```

2565         ↪ "where we left", "yes", "yeah", "yep")
2566     if any(k in low for k in fresh_kw):
2567         intent = "FRESH"
2568     elif any(k in low for k in continue_kw):
2569         intent = "CONTINUE"
2570     else:
2571         # ambiguous; preserve the save rather than wipe it
2572         intent = "CONTINUE"
2573     print(f" Keyword fallback resolved intent: {intent}")
2574 if "CONTINUE" in intent:
2575     engine = restore_engine(save_data)
2576     preferred_game = engine.current_game
2577     restored_rounds = save_data.get("total_rounds_played",
2578                                     save_data.get("rounds_played", 0))
2579     restore_msg = f"Restoring your previous session: {restored_rounds} rounds on record."
2580     say(ssh_tts, restore_msg)
2581     print(f"\nRobot: {restore_msg}")
2582 else:
2583     delete_save()
2584     fresh_msg = "No worries, starting fresh! Your previous save has been cleared."
2585     say(ssh_tts, fresh_msg)
2586     print(f"\nRobot: {fresh_msg}")
2587
2588 if not LOCAL_MODE:
2589     nao_track_face(ssh, enable=True)
2590     nao_set_leds(ssh, "FaceLeds", 0x0000FF00, 1.0)
2591
2592 conversation = [{"role": "system", "content": BASE_SYSTEM_PROMPT}]
2593
2594 greeting_directive = "[Give a warm, supportive greeting. Mention you can chat, play games, or
2595 ↪ just hang out. Keep it to 2 sentences.]"
2596 greeting_msg = converse(
2597     conversation + [{"role": "user", "content": greeting_directive}],
2598     TOOLS,
2599 )
2600 greeting_text = greeting_msg.content or "Hello, lovely to meet you!"
2601 greeting_gesture = extract_gesture(greeting_text)
2602 greeting_speech = re.sub(r'\[gesture:\w+\]', '', greeting_text).strip()
2603
2604 conversation.append({"role": "assistant", "content": greeting_text})
2605
2606 if not LOCAL_MODE:
2607     nao_gesture(ssh, "wave")
2608     say(ssh_tts, greeting_speech)
2609     print(f"\nRobot: {greeting_speech}")
2610     dashboard.update_robot_speech(greeting_speech)
2611
2612 turn_count = 0
2613 # silent turns skip the LLM so the robot doesn't pester. Tiered disengagement: tier 1 check-in
2614 ↪ at 3 silences, tier 2 game-switch at 4; resets when user speaks
2615 nudge_level = 0
2616
2617 try:
2618     while True:
2619         turn_count += 1
2620         print(f"\n{'-' * 40} Turn {turn_count} {'-' * 40}")

```

```

2621     print("Capturing expression...")
2622     expression, expr_conf = capture_and_classify(
2623         ssh, face_model, face_cascade, local_camera
2624     )
2625     if expr_conf < FACE_CONFIDENCE_THRESHOLD:
2626         print(f" Expression: {expression} ({expr_conf:.2f}): LOW CONFIDENCE, treating as
                ↳ Neutral")
2627         expression = "Neutral"
2628     else:
2629         print(f" Expression: {expression} ({expr_conf:.2f})")
2630
2631     question_start = time.time()
2632     print("Listening...")
2633     if not LOCAL_MODE:
2634         # brief cyan pulse on the face LEDs so the user can see the robot is actively
                ↳ listening; ears also go green
2635         nao_set_leds(ssh, "FaceLeds", 0x0000FFFF, 0.2)
2636         nao_set_leds(ssh, "EarLeds", 0x0000FF00, 0.3)
2637
2638         # adaptive think-budget based on previous round's state + current face
2639         prev_state = (engine.history[-1].inferred_state if engine.history
2640                     else InferredState.COMFORTABLE)
2641         prev_rt = engine.history[-1].response_time if engine.history else 0.0
2642         no_speech_max, silence_secs, record_max_secs = engine.recommend_think_budget(
2643             state=prev_state, expression=expression,
2644             prev_response_time=prev_rt,
2645             consecutive_silences=engine.consecutive_silences,
2646             waiting=game_state.waiting,
2647         )
2648         dashboard.update_think_budget(record_max_secs)
2649
2650         record(ssh, energy_threshold,
2651             no_speech_max=no_speech_max,
2652             silence_secs=silence_secs,
2653             record_max_secs=record_max_secs)
2654         response_time = time.time() - question_start
2655
2656         # extended think-budget consumed; reset for next turn
2657         if game_state.waiting:
2658             game_state.waiting = False
2659
2660         vocal_emo, vocal_conf = classify_speech_emotion(speech_model, LOCAL_WAV)
2661         if vocal_conf < VOICE_CONFIDENCE_THRESHOLD:
2662             print(f" Vocal emotion: {vocal_emo} ({vocal_conf:.2f}): LOW CONFIDENCE, treating as
                    ↳ neutral")
2663             vocal_emo = "neutral"
2664         else:
2665             print(f" Vocal emotion: {vocal_emo} ({vocal_conf:.2f})")
2666
2667         vol_rms = measure_volume()
2668         print(f" Volume RMS: {vol_rms:.0f}")
2669
2670         # (c) transcribe; per-chunk gate, file-wide RMS removed
2671         relisten = lambda: record(ssh, energy_threshold,
2672             no_speech_max=no_speech_max,
2673             silence_secs=silence_secs,
2674             record_max_secs=record_max_secs)
2675         if INPUT_IS_LOCAL: # hybrid check
2676             if _local_speech_detected:

```

```

2677         user_text = transcribe(bypass_wake_word=True, record_again=relisten)
2678     else:
2679         print(f" No speech chunk detected (floor={LOCAL_SILENCE_RMS}); skipping
           ↳ transcription.")
2680         user_text = ""
2681 elif vol_rms >= NAO_MIN_RMS_TO_TRANSCRIBE:
2682     print(f" NAO RMS diagnostic: {vol_rms:.0f} (gate floor {NAO_MIN_RMS_TO_TRANSCRIBE})")
2683     user_text = transcribe(bypass_wake_word=True, record_again=relisten)
2684 else:
2685     print(f" WAV near-empty ({vol_rms:.0f} < {NAO_MIN_RMS_TO_TRANSCRIBE}); skipping
           ↳ transcription.")
2686     user_text = ""
2687
2688 if user_text:
2689     print(f" Heard: {user_text}")
2690     dashboard.append_user_speech(user_text)
2691     # user spoke; reset silence + nudge so future silence re-arms from scratch
2692     engine.consecutive_silences = 0
2693     nudge_level = 0
2694
2695     if is_goodbye(user_text):
2696         dashboard.quit_app()
2697 else:
2698     print(" Heard: (silence)")
2699     dashboard.append_user_speech("(silence)")
2700     engine.consecutive_silences += 1
2701
2702     # surface signals to dashboard even though the LLM is skipped this turn
2703     dashboard.update_signals(
2704         round_num=turn_count, user_answer="", correct_answer="",
2705         correct=False, expression=expression, expr_conf=expr_conf,
2706         vocal_emo=vocal_emo, vocal_conf=vocal_conf,
2707         vol_rms=vol_rms, response_time=response_time,
2708         rolling_acc=engine.rolling_correctness(),
2709         total_correct=engine.total_correct,
2710         total_rounds=engine.total_rounds_played,
2711         streak=engine.consecutive_correct,
2712     )
2713
2714     # 1- gentle check-in at 3 consecutive silences; nudge_level guards each tier to once
2715     ↳ per silent spell (proposal: "intervenes to re-engage them",
2716     ↳ assistive/stroke-rehab analogue)
2717 if engine.consecutive_silences >= 3 and nudge_level < 1:
2718     nudge_prompt = [
2719         {"role": "system",
2720          "content": BASE_SYSTEM_PROMPT},
2721         {"role": "user",
2722          "content": "[The user has been quiet for 3 turns. Offer ONE brief, gentle
2723          ↳ check-in; no question, no nagging. 1 sentence max.]"},
2724     ]
2725     nudge_msg = converse(nudge_prompt, [])
2726     nudge_text = (nudge_msg.content or "").strip()
2727     nudge_speech = re.sub(r'\[gesture:\w+\]', '', nudge_text).strip()
2728     if not LOCAL_MODE:
2729         # recolour eyes to signal the state has shifted; user sees the shift even if
2730         ↳ they say nothing back
2731         nao_set_leds(ssh, "FaceLeds",
2732                     LED_COLOURS[InferredState.DISENGAGED], 0.4)
2733     if nudge_speech:

```

```

2730         say(ssh_tts, nudge_speech)
2731         print(f"\nRobot (nudge): {nudge_speech}")
2732         dashboard.update_robot_speech(nudge_speech)
2733         nudge_level = 1
2734
2735     # 2- switch game, soft re-invitation
2736     elif engine.consecutive_silences >= 4 and nudge_level < 2:
2737         engine.current_game = engine.pick_different_game()
2738         engine.game_switch_count += 1
2739         escalate_prompt = [
2740             {"role": "system",
2741              "content": BASE_SYSTEM_PROMPT},
2742             {"role": "user",
2743              "content": f"[The user has gone silent for 4 turns. Warmly acknowledge
                ↪ they've gone quiet then offer a {engine.current_game.value} round as a
                ↪ soft alternative. Two sentences total.]"},
2744         ]
2745         esc_msg = converse(escalate_prompt, [])
2746         esc_text = (esc_msg.content or "").strip()
2747         esc_speech = re.sub(r'\[gesture:\w+\]', '', esc_text).strip()
2748         if not LOCAL_MODE:
2749             nao_set_leds(ssh, "FaceLeds",
2750                          LED_COLOURS[InferredState.DISENGAGED], 0.4)
2751         if esc_speech:
2752             say(ssh_tts, esc_speech)
2753             print(f"\nRobot (escalate): {esc_speech}")
2754             dashboard.update_robot_speech(esc_speech)
2755             nudge_level = 2
2756
2757         continue # skip the LLM call; just wait for the user
2758
2759     if user_text.lower().strip() in [
2760         "stop", "quit", "exit", "goodbye", "bye", "end",
2761         "i want to stop", "let's stop", "no more",
2762     ]:
2763         print("Patient wants to stop.")
2764         break
2765
2766     signal_ctx = build_signal_context(
2767         engine, expression, expr_conf,
2768         vocal_emo, vocal_conf, vol_rms, response_time,
2769     )
2770
2771     user_msg_content = f"{signal_ctx}\n\nUser says: {user_text}"
2772     conversation.append({"role": "user", "content": user_msg_content})
2773
2774     # trim conversation to prevent context overflow; keep system + last 40 messages (20
2775     ↪ exchanges)
2776     if len(conversation) > 42:
2777         conversation = [conversation[0]] + conversation[-40:]
2778
2779     if not LOCAL_MODE:
2780         nao_set_leds(ssh, "EarLeds", 0x000000FF, 0.3) # blue = thinking
2781
2782     llm_message = converse(conversation, TOOLS)
2783
2784     response_text = process_llm_response(
2785         llm_message, conversation, engine, game_state,
2786         preferred_game, dashboard

```

```

2786     )
2787
2788     if not response_text.strip():
2789         response_text = "I'm here! What would you like to talk about? [gesture:neutral]"
2790
2791     gesture_type = extract_gesture(response_text)
2792     speech_text = re.sub(r'\[gesture:\w+\]', '', response_text).strip()
2793
2794     if not LOCAL_MODE:
2795         if engine.adaptation_log:
2796             last_state_str = engine.adaptation_log[-1]["state"]
2797             try:
2798                 last_state = InferredState(last_state_str)
2799             except ValueError:
2800                 last_state = InferredState.COMFORTABLE
2801             nao_set_leds(
2802                 ssh, "FaceLeds",
2803                 LED_COLOURS.get(last_state, 0x00FFFFFF), 0.5
2804             )
2805
2806     say(ssh_tts, speech_text)
2807
2808     print(f"\nRobot: {speech_text}")
2809     if game_state.current_answer:
2810         print(f"(Game answer: {game_state.current_answer})")
2811     dashboard.update_robot_speech(speech_text)
2812
2813     # (k) update dashboard (signals + decision if game active); use the actual answer check
2814     ↳ result from the tool chain
2815     was_game_answer = game_state.last_answer_checked
2816     correct = game_state.last_answer_correct if was_game_answer else False
2817
2818     if not LOCAL_MODE and was_game_answer and correct:
2819         threading.Thread(target=nao_gesture, args=(ssh, "correct_wave"), daemon=True).start()
2820
2821     # only run engine.decide on real game answers; chat or more-time turns mustn't ramp
2822     ↳ difficulty or pollute streak counters
2823     if was_game_answer:
2824         # use snapshot so a same-chain generate_game_question can't pollute this round's
2825         ↳ logged values
2826         answered_q = game_state.answered_question or game_state.current_question
2827         answered_a = game_state.answered_answer or game_state.current_answer
2828         answered_gt = game_state.answered_game_type or engine.current_game
2829         answered_df = game_state.answered_difficulty or engine.current_difficulty
2830         decision = engine.decide(
2831             expression, expr_conf, response_time,
2832             correct=correct,
2833             answer_text=user_text,
2834             vocal_emotion=vocal_emo, vocal_conf=vocal_conf,
2835             volume_rms=vol_rms,
2836         )
2837         engine.record_round(RoundResult(
2838             round_number=turn_count,
2839             game_type=answered_gt,
2840             difficulty=answered_df,
2841             question=answered_q,
2842             user_answer=user_text,
2843             correct=correct,
2844             response_time=response_time,

```

```

2842         facial_expression=expression,
2843         expression_confidence=expr_conf,
2844         vocal_emotion=vocal_emo,
2845         vocal_emotion_confidence=vocal_conf,
2846         volume_rms=vol_rms,
2847         inferred_state=decision.inferred_state,
2848     ))
2849     # save after every round; power-cycle loses one turn, not five
2850     save_session(engine, preferred_game, quiet=True)
2851     dashboard.update_signals(
2852         round_num=turn_count, user_answer=user_text,
2853         correct_answer=answered_a,
2854         correct=correct,
2855         expression=expression, expr_conf=expr_conf,
2856         vocal_emo=vocal_emo, vocal_conf=vocal_conf,
2857         vol_rms=vol_rms, response_time=response_time,
2858         rolling_acc=engine.rolling_correctness(),
2859         total_correct=engine.total_correct,
2860         total_rounds=engine.total_rounds_played,
2861         streak=engine.consecutive_correct,
2862     )
2863     dashboard.update_decision(decision)
2864     # clear snapshot now the round is logged
2865     game_state.answered_question = ""
2866     game_state.answered_answer = ""
2867     game_state.answered_game_type = None
2868     game_state.answered_difficulty = None
2869     else:
2870         # chat or more-time turn; refresh dashboard signals only
2871         dashboard.update_signals(
2872             round_num=turn_count, user_answer=user_text,
2873             correct_answer="",
2874             correct=False,
2875             expression=expression, expr_conf=expr_conf,
2876             vocal_emo=vocal_emo, vocal_conf=vocal_conf,
2877             vol_rms=vol_rms, response_time=response_time,
2878             rolling_acc=engine.rolling_correctness(),
2879             total_correct=engine.total_correct,
2880             total_rounds=engine.total_rounds_played,
2881             streak=engine.consecutive_correct,
2882         )
2883
2884     game_state.last_answer_checked = False
2885
2886     # belt-and-braces auto-save: even on quiet/chat turns where no round was recorded, flush
2887     ↳ progress every 2 turns so mid-conversation state (name, recent_questions,
2888     ↳ streaks) is never stale by more than one turn
2889     if turn_count % 2 == 0:
2890         save_session(engine, preferred_game, quiet=True)
2891         print(" [Auto-save]")
2892
2893 except KeyboardInterrupt:
2894     print("\n\nInterrupted.")
2895
2896 save_session(engine, preferred_game)
2897
2898 summary = engine.get_session_summary()
2899 print(f"\n{'=' * 60}")

```

```

2899     print(" SESSION SUMMARY")
2900     print(f"{'=' * 60}")
2901     for k, v in summary.items():
2902         print(f" {k}: {v}")
2903
2904     if summary.get("rounds", 0) > 0:
2905         acc = summary["accuracy"]
2906         streak_note = (f" Your best streak was {summary['best_streak']} in a row!"
2907                        if summary["best_streak"] >= 3 else "")
2908         if acc >= 0.8:
2909             farewell = f"Amazing session! You got {summary['correct']} out of {summary['rounds']}
2910                        ↪ right.{streak_note} Brilliant work! Your progress is saved; see you next time!"
2911             elif acc >= 0.5:
2912                 farewell = f"Great effort! You scored {summary['correct']} out of
2913                            ↪ {summary['rounds']}.{streak_note} Well played! Your progress is saved; see you
2914                            ↪ next time!"
2915             else:
2916                 farewell = f"Thanks for playing! You got {summary['correct']} out of
2917                            ↪ {summary['rounds']}.{streak_note} Every round is a learning opportunity. Your
2918                            ↪ progress is saved; see you next time!"
2919         else:
2920             farewell = "It was great to chat! Your progress is saved; see you next time!"
2921
2922     if not LOCAL_MODE:
2923         nao_gesture(ssh, "wave")
2924         say(ssh_tts, farewell)
2925         print(f"\nRobot: {farewell}")
2926
2927     if local_camera is not None:
2928         local_camera.release()
2929     if not LOCAL_MODE:
2930         nao_track_face(ssh, enable=False)
2931         nao_set_leds(ssh, "FaceLeds", 0x00000000, 0.5)
2932         ssh.close()
2933         ssh_tts.close()
2934         dashboard.close()
2935         print("\nGAZE disconnected.")
2936
2937 def main():
2938     print("-----")
2939     print(" GAZE: Game-Adaptive Zone of Engagement")
2940     print(" Adaptive Game-System Ran on Pepper")
2941     print("-----")
2942
2943     print("\nLoading facial expression model...")
2944     face_model = FacialExpressionModel(MODEL_JSON, MODEL_WEIGHTS)
2945     face_cascade = cv2.CascadeClassifier(HAAR_CASCADE)
2946     print(" Facial model loaded.")
2947
2948     speech_model = None
2949     if os.path.exists(SPEECH_MODEL):
2950         print("Loading speech emotion model...")
2951         speech_model = SpeechEmotionModel(SPEECH_MODEL)
2952         print(" Speech model loaded.")
2953     else:
2954         print(f" Speech emotion model not found at {SPEECH_MODEL}; vocal signal disabled.")
2955         print(" Run train_speech_model.py to generate it.")
2956
2957     local_camera = None

```



```

2953 if USE_LOCAL_CAMERA:
2954     local_camera = cv2.VideoCapture(0)
2955     # Set capture properties explicitly; `BUFFERSIZE=1` is the critical one: without it OpenCV
        ↳ buffers 3-5 frames and `camera.read()` pulls stale ones, compounding perceived
        ↳ latency even when every downstream stage is fast.
2956     local_camera.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
2957     local_camera.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
2958     local_camera.set(cv2.CAP_PROP_FPS, 30)
2959     local_camera.set(cv2.CAP_PROP_BUFFERSIZE, 1)
2960     print(" Using local webcam for expression detection.")
2961     if DEBUG_PREVIEW:
2962         start_preview_thread(local_camera, face_model, face_cascade)
2963         print(" Preview thread started.")
2964     else:
2965         print(" Using Pepper's camera for expression detection.")
2966
2967     # connect to Pepper (skipped in local mode)
2968     ssh = None
2969     ssh_tts = None
2970     energy_threshold = DEFAULT_ENERGY_THRESHOLD
2971
2972     # Pepper SSH connection: required whenever output goes through the robot
2973     # (TTS, LEDs, gestures, face-tracking). LOCAL_MODE remains the gate for that.
2974     if LOCAL_MODE:
2975         print("\n LOCAL MODE: skipping Pepper connection.")
2976     else:
2977         print(f"\nConnecting to Pepper at {NAO_IP}...")
2978         ssh = ssh_connect()
2979         ssh_tts = ssh_connect()    # dedicated TTS connection
2980         print(" Connected.")
2981
2982     # Pepper stream ~10 fps; detect at 6-7 Hz
2983     if not USE_LOCAL_CAMERA:
2984         print(" Starting Pepper camera stream...")
2985         threading.Thread(target=pepper_video_loop,
2986                         args=(ssh,), daemon=True).start()
2987         threading.Thread(target=pepper_camera_receive_loop,
2988                         args=(NAO_IP,), daemon=True).start()
2989         threading.Thread(target=detect_thread_loop,
2990                         args=(face_model, face_cascade), daemon=True).start()
2991         print(" Pepper camera stream threads started.")
2992
2993     # Mic-side ambient calibration: pick whichever microphone INPUT_IS_LOCAL says.
2994     # In gaze21 hybrid mode this is the Mac mic even though Pepper is connected.
2995     if INPUT_IS_LOCAL:
2996         print("\nCalibrating Mac microphone ambient noise level...")
2997         local_calibrate_ambient()
2998     else:
2999         print("\nCalibrating ambient noise level (stay quiet for 3 seconds)...")
3000         energy_threshold = nao_calibrate_ambient(ssh)
3001
3002     dashboard = GazeDashboard(ssh=ssh, ssh_tts=ssh_tts)
3003     print(" Dashboard launched.")
3004
3005     conv_thread = threading.Thread(
3006         target=conversation_loop,
3007         args=(dashboard, face_model, face_cascade, speech_model,
3008             local_camera, ssh, ssh_tts, energy_threshold),
3009         daemon=True,

```

```
3010     )
3011     conv_thread.start()
3012
3013     # tkinter mainloop on main thread (macOS-required); keep camera preview and GUI responsive
3014     ↳ whilst the conversation loop blocks on I/O
3015     dashboard.root.mainloop()
3016
3017 if __name__ == "__main__":
3018     main()
```